Java Senior Interview Guide

Questions & Explained Answers for 9+ Years Experience

Core Java & Java 8+ Features
Collections Framework (In Depth)
Spring & Spring Boot
SQL & Relational Databases
NoSQL Databases
Microservices Architecture & Patterns
Design Principles & Design Patterns
System Design
Apache Kafka
Azure Cloud
DevOps, CI/CD, Docker & Kubernetes
Senior-Level Scenario & Coding Questions

171 questions with explanations, code samples and interview tips

Contents

1. Core Java & Java 8+ Features 7

Q1. What changed in Java 8 and why does it matter for a senior developer? 7
Q2. What happened after Java 8 (Java 9 to 21+)? Which features have you used? 7
Q3. Explain functional interfaces and the main built-in ones. 8
Q4. How do Streams work internally? What are intermediate and terminal operations? 8
Q5. map vs flatMap? reduce vs collect? 8
Q6. When should you NOT use parallel streams? 9
Q7. How should Optional be used correctly? 9
Q8. Explain JVM memory areas. 9
Q9. How does Garbage Collection work? Compare G1, ZGC and Parallel GC. 9
Q10. What is the Java Memory Model? Explain volatile, happens-before and visibility. 10
Q11. synchronized vs ReentrantLock vs atomic classes? 10
Q12. Explain the ExecutorService, thread pools and how you size them. 10
Q13. How does CompletableFuture work? Give a real example. 11
Q14. What are virtual threads and when would you use them? 11
Q15. What causes deadlock and how do you detect and prevent it? 11
Q16. Why must you override equals() and hashCode() together? 12
Q17. How do you design an immutable class? 12
Q18. Why is String immutable? String pool, StringBuilder vs StringBuffer? 12
Q19. Checked vs unchecked exceptions? Best practices? 12
Q20. Difference between abstract class and interface after Java 8? 13
Q21. Explain ClassLoaders and how they delegate. 13
Q22. What is serialization? What are its risks? 13
Q23. How do you troubleshoot a slow or high-CPU Java service in production? 13

2. Collections Framework (In Depth) 14

Q24. Explain the Collections Framework hierarchy. 14
Q25. How does HashMap work internally? (The most-asked collections question) 14
Q26. HashMap vs Hashtable vs ConcurrentHashMap vs Collections.synchronizedMap? 14
Q27. How does ConcurrentHashMap achieve thread safety? 15
Q28. ArrayList vs LinkedList? When would you use each? 15
Q29. HashSet, LinkedHashSet and TreeSet: how are they implemented? 15
Q30. How does LinkedHashMap work? Build an LRU cache. 15
Q31. TreeMap internals and when to use it? 16
Q32. Comparable vs Comparator? 16
Q33. Fail-fast vs fail-safe iterators? 16
Q34. What is CopyOnWriteArrayList and when should you use it? 16
Q35. Explain BlockingQueue implementations and the producer-consumer pattern. 17
Q36. PriorityQueue: how does it work? Solve "top K". 17

Q37. WeakHashMap, IdentityHashMap and EnumMap: use cases? . . . 17
Q38. Unmodifiable vs immutable collections? . . . 17
Q39. Big-O cheat sheet for common collections. . . . 17
Q40. Can a mutable object be used as a HashMap key? What happens? . . . 18

3. Spring & Spring Boot . . . 19

Q41. What is Spring Boot and how does auto-configuration work? . . . 19
Q42. Explain IoC and Dependency Injection. Constructor vs field injection? . . . 19
Q43. Explain the Spring Bean lifecycle. . . . 19
Q44. Bean scopes? What happens when you inject a prototype into a singleton? . . . 19
Q45. How does @Transactional work? What are the common pitfalls? . . . 20
Q46. Explain transaction propagation and isolation levels. . . . 20
Q47. What is the N+1 query problem in JPA/Hibernate and how do you fix it? . . . 20
Q48. Optimistic vs pessimistic locking? . . . 21
Q49. How do you handle exceptions globally in a REST API? . . . 21
Q50. How does Spring AOP work? Where have you used it? . . . 21
Q51. How do you manage configuration across environments? . . . 21
Q52. What does Spring Boot Actuator give you? . . . 22
Q53. How do you secure a Spring Boot microservice (JWT / OAuth2)? . . . 22
Q54. How do you test Spring Boot applications? . . . 22
Q55. How does Spring's REST client story look today (RestTemplate, WebClient, RestClient)? . . . 22
Q56. Spring MVC vs WebFlux? When would you choose reactive? . . . 23
Q57. How do you implement caching in Spring Boot? . . . 23
Q58. How do you improve Spring Boot startup time and memory? . . . 23

4. SQL & Relational Databases . . . 24

Q59. What are ACID properties? Explain each with an example. . . . 24
Q60. Explain the types of JOINs. . . . 24
Q61. WHERE vs HAVING? What is the logical order of SQL execution? . . . 24
Q62. Find the Nth highest salary. (Classic question, three ways.) . . . 25
Q63. Explain window functions. ROW_NUMBER vs RANK vs DENSE_RANK? . . . 25
Q64. How do you find and delete duplicate rows? . . . 25
Q65. What are CTEs and recursive CTEs? . . . 25
Q66. Subquery vs JOIN vs EXISTS? IN vs EXISTS? . . . 26
Q67. DELETE vs TRUNCATE vs DROP? . . . 26
Q68. How do indexes work? Clustered vs non-clustered? . . . 26
Q69. What is a composite index and the leftmost prefix rule? . . . 26
Q70. Why might a query NOT use an index? . . . 26
Q71. How do you optimise a slow SQL query? . . . 27
Q72. Explain normalization (1NF, 2NF, 3NF, BCNF). When do you denormalize? . . . 27
Q73. Explain concurrency anomalies and isolation levels in the database. . . . 27
Q74. What causes database deadlocks and how do you handle them? . . . 27

Q75. What are views, materialized views, stored procedures and triggers? . . . 28
Q76. What is database partitioning and how is it different from sharding? . . . 28
Q77. How do you manage database schema changes in microservices? . . . 28
Q78. Practical SQL questions to practise. . . . 29
Q79. How do you size and tune a JDBC connection pool (HikariCP)? . . . 29

5. NoSQL Databases . . . 30

Q80. What is NoSQL and what types exist? . . . 30
Q81. When would you choose NoSQL over SQL? . . . 30
Q82. What is BASE and eventual consistency? . . . 30
Q83. How do you model data in a document database (MongoDB / Cosmos DB)? . . . 30
Q84. How do you choose a partition key in Cosmos DB (or a shard key in general)? . . . 31
Q85. Explain Cosmos DB key concepts: RU/s, consistency levels and change feed. . . . 31
Q86. How does Cassandra work? What is its data model? . . . 31
Q87. What is Redis and what are its main use cases and data structures? . . . 32
Q88. How do you implement a distributed lock? What are the pitfalls? . . . 32
Q89. How does Elasticsearch work and when would you use it? . . . 32
Q90. How do you keep data consistent between a SQL database and a NoSQL / search store? . . . 33
Q91. Rapid-fire SQL / NoSQL questions. . . . 33

6. Microservices Architecture & Patterns . . . 34

Q92. Monolith vs microservices: when would you NOT use microservices? . . . 34
Q93. How do you decide service boundaries? (Domain-Driven Design) . . . 34
Q94. Explain the API Gateway pattern. . . . 34
Q95. Service discovery: client-side vs server-side? . . . 34
Q96. Synchronous vs asynchronous communication between services? . . . 35
Q97. Explain the Circuit Breaker pattern (Resilience4j). . . . 35
Q98. How do you handle distributed transactions? Explain the Saga pattern. . . . 35
Q99. What is the Transactional Outbox pattern and why do you need it? . . . 35
Q100. Explain CQRS and Event Sourcing. . . . 36
Q101. How do you make APIs and consumers idempotent? . . . 36
Q102. Database per service: how do you run queries that span services? . . . 36
Q103. How do you implement observability in microservices? . . . 36
Q104. How do you version APIs and avoid breaking consumers? . . . 37
Q105. What is the Strangler Fig pattern? How would you migrate a monolith? . . . 37
Q106. What other microservice patterns should you know? . . . 37
Q107. How do you secure service-to-service communication? . . . 37

7. Design Principles & Design Patterns . . . 38

Q108. Explain the SOLID principles with examples. . . . 38
Q109. Singleton: how do you implement it correctly? What are its problems? . . . 38
Q110. Factory Method vs Abstract Factory? . . . 38

Q111. Builder pattern: when and why? . . . 39
Q112. Strategy pattern: give a real-world example. . . . 39
Q113. Observer pattern and how it relates to event-driven systems? . . . 39
Q114. Decorator vs Proxy vs Adapter? . . . 39
Q115. Template Method vs Strategy? . . . 39
Q116. Chain of Responsibility: where is it used? . . . 40
Q117. Command, State and Template patterns in business workflows? . . . 40
Q118. Which design patterns does the Spring Framework use? . . . 40
Q119. What are anti-patterns you have seen and fixed? . . . 40
Q120. Explain Hexagonal / Clean Architecture. . . . 40

8. System Design . . . 41

Q121. How do you approach a system design interview? (Framework) . . . 41
Q122. Explain the CAP theorem and PACELC. . . . 41
Q123. SQL vs NoSQL: how do you choose? . . . 41
Q124. How do you scale a database? . . . 41
Q125. Explain caching strategies and cache invalidation. . . . 42
Q126. Design a URL shortener (like bit.ly). . . . 42
Q127. Design a rate limiter. . . . 42
Q128. Design a notification system (email / SMS / push). . . . 43
Q129. Design an e-commerce order and payment system. . . . 43
Q130. Design a chat / messaging system (like WhatsApp). . . . 43
Q131. How do you design for high availability and disaster recovery? . . . 43
Q132. How do you generate unique IDs in a distributed system? . . . 44
Q133. Explain load balancing algorithms and L4 vs L7. . . . 44
Q134. How do you handle backpressure and traffic spikes? . . . 44

9. Apache Kafka . . . 45

Q135. Explain Kafka's architecture. . . . 45
Q136. How do partitions and consumer groups work? How does Kafka guarantee ordering? . . . 45
Q137. Explain delivery semantics: at-most-once, at-least-once, exactly-once. . . . 45
Q138. Which producer configurations matter for reliability? . . . 45
Q139. Explain consumer offsets, commits and rebalancing. . . . 46
Q140. How do you handle failures and poison messages? (Retry + DLT) . . . 46
Q141. What is consumer lag and how do you fix a growing lag? . . . 46
Q142. Retention vs log compaction? . . . 47
Q143. What is Schema Registry and why use Avro or Protobuf? . . . 47
Q144. Kafka vs RabbitMQ vs Azure Service Bus vs Event Hubs? . . . 47
Q145. What is Kafka Streams / Kafka Connect? . . . 47
Q146. How do you produce and consume with Spring Kafka? . . . 48
Q147. How would you size a Kafka topic (partitions)? . . . 48

10. Azure Cloud 49

Q148. Which Azure compute options would you choose for Java microservices? 49
Q149. How do you manage secrets and identity in Azure? 49
Q150. Azure Service Bus vs Event Hubs vs Event Grid vs Storage Queues? 49
Q151. How do you choose a database on Azure? 49
Q152. What is Azure API Management and where does it fit? 50
Q153. How do you monitor Java apps on Azure? 50
Q154. How do you design a secure network in Azure? 50
Q155. How do you autoscale on Azure? 50

11. DevOps, CI/CD, Docker & Kubernetes 51

Q156. Describe a CI/CD pipeline you have built for Java microservices. 51
Q157. How do you write a production-grade Dockerfile for a Spring Boot app? 51
Q158. Explain core Kubernetes objects. 52
Q159. Explain liveness, readiness and startup probes. 52
Q160. Deployment strategies: rolling, blue-green, canary? 52
Q161. What is Infrastructure as Code? Terraform vs Bicep? 52
Q162. What is GitOps? 53
Q163. What DevOps metrics do you track? (DORA) 53
Q164. A pod is in CrashLoopBackOff. How do you debug it? 53

12. Senior-Level Scenario & Coding Questions 54

Q165. Write a thread-safe, generic, bounded producer-consumer without BlockingQueue. 54
Q166. Common Stream coding questions (write them live). 54
Q167. Production incident: latency spikes every few minutes. How do you investigate? 55
Q168. Your service must handle 10x traffic next month. What do you do? 55
Q169. How do you ensure data consistency between the Order service DB and Kafka events? 55
Q170. How do you mentor developers and drive technical decisions? (Leadership) 55
Q171. Rapid-fire questions to revise the night before. 55

1. Core Java & Java 8+ Features

Q1. What changed in Java 8 and why does it matter for a senior developer?

Java 8 moved Java towards a **functional style**. The key additions are:

- **Lambda expressions**: short anonymous functions, e.g. `(a, b) -> a + b`.
- **Functional interfaces**: interfaces with exactly one abstract method (`Function`, `Predicate`, `Supplier`, `Consumer`, `BiFunction`, `UnaryOperator`).
- **Stream API**: declarative pipelines over data (filter, map, reduce) that can run in parallel.
- **Optional**: an explicit container for "value may be absent", which reduces NullPointerExceptions.
- **Default and static methods in interfaces**: APIs can grow without breaking existing implementations.
- **java.time API**: immutable, thread-safe dates (`LocalDate`, `ZonedDateTime`, `Instant`, `Duration`).
- **CompletableFuture**: composable asynchronous programming.
- **Method references**: `String::toUpperCase`, `System.out::println`.
- **Metaspace** replaced PermGen for class metadata.

> **Interview tip:** Senior angle: don't just list features. Say how they changed your code, for example "we replaced nested loops over DTOs with streams and Collectors.groupingBy, and made our service layer return Optional instead of null".

Q2. What happened after Java 8 (Java 9 to 21+)? Which features have you used?

- **Java 9**: Module system (JPMS), `List.of()` / `Map.of()` immutable factories, JShell, private interface methods.
- **Java 10**: `var` for local variable type inference.
- **Java 11 (LTS)**: `HttpClient`, the String methods `isBlank()`, `strip()`, `lines()` and `repeat()`, and running single-file programs with `java File.java`.
- **Java 14-16**: Switch expressions, **Records** (immutable data carriers), pattern matching for `instanceof`, text blocks.
- **Java 17 (LTS)**: **Sealed classes** (`sealed ... permits`), which limit which classes can extend a type.
- **Java 21 (LTS)**: **Virtual threads** (Project Loom), pattern matching for switch, record patterns, sequenced collections (`getFirst()`, `getLast()`, `reversed()`).

```
// Record + sealed interface + pattern matching switch (Java 21)
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Square sq -> sq.side() * sq.side();
    };   // no default needed: the compiler knows every subtype
}
```

Q3. Explain functional interfaces and the main built-in ones.

A functional interface has **exactly one abstract method** (SAM). It can still have default and static methods. You can mark it with `@FunctionalInterface` so the compiler enforces the rule.

- `Predicate<T>`: T -> boolean (used by `filter`)
- `Function<T,R>`: T -> R (used by `map`)
- `Consumer<T>`: T -> void (used by `forEach`)
- `Supplier<T>`: () -> T (lazy creation, `orElseGet`)
- `BiFunction<T,U,R>`, `UnaryOperator<T>`, `BinaryOperator<T>`
- Primitive specialisations (`IntPredicate`, `ToIntFunction`) avoid boxing overhead.

```
Predicate<String> notEmpty = s -> !s.isEmpty();
Function<String, Integer> len = String::length;
Function<String, Integer> lenTimes2 = len.andThen(x -> x * 2);
Predicate<String> valid = notEmpty.and(s -> s.length() < 50);
```

Q4. How do Streams work internally? What are intermediate and terminal operations?

A stream is a **lazy pipeline**. It has a source, zero or more **intermediate operations** (filter, map, sorted, distinct, flatMap, peek) and one **terminal operation** (collect, forEach, reduce, count, findFirst, anyMatch).

- Nothing runs until the terminal operation is called.
- Elements are processed **one at a time through the whole pipeline** (vertical processing), not stage by stage. This allows **short-circuiting**: `findFirst()` stops as soon as one element matches.
- **Stateless** operations (map, filter) are cheap. **Stateful** ones (sorted, distinct) must buffer elements.
- A stream **cannot be reused** after its terminal operation. Doing so throws `IllegalStateException`.

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .filter(e -> e.getSalary() > 50_000)
    .collect(Collectors.groupingBy(Employee::getDept,
             Collectors.averagingDouble(Employee::getSalary)));

// Top 3 highest paid names
List<String> top3 = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .limit(3).map(Employee::getName).toList();
```

Q5. map vs flatMap? reduce vs collect?

- **map** transforms each element one-to-one: `Stream<List<String>>` stays a stream of lists.
- **flatMap** turns each element into a stream and **flattens** them: `Stream<List<String>>` becomes `Stream<String>`.
- **reduce** combines elements into one immutable value (sum, max). Use it for immutable reduction.
- **collect** is a **mutable reduction** into a container (List, Map, StringBuilder). It is more efficient for building collections.

```
List<String> allSkills = employees.stream()
    .flatMap(e -> e.getSkills().stream()).distinct().toList();
int total = orders.stream().map(Order::getQty).reduce(0, Integer::sum);
```

Q6. When should you NOT use parallel streams?

Parallel streams use the **common ForkJoinPool**, which has (number of cores - 1) threads and is shared by the whole JVM.

- **Avoid** them for I/O or blocking work (DB, REST calls). They block the shared pool and slow down the whole application.
- **Avoid** them for small datasets. Splitting and merging costs more than you gain.
- **Avoid** them with sources that split badly (`LinkedList`, `Stream.iterate`) or with order-dependent operations (`findFirst`, `limit` on ordered streams).
- **Avoid** them with shared mutable state. That causes race conditions.
- **Good fit**: large, CPU-bound, stateless work over `ArrayList` or arrays.

Q7. How should Optional be used correctly?

- Use Optional as a **return type** to show a value may be absent, e.g. `Optional<User> findById(id)`.
- **Do not** use it for fields, method parameters or collections (return an empty list instead). Optional is not Serializable.
- Prefer `orElseGet(() -> expensive())` over `orElse(expensive())`. `orElse` **always** evaluates its argument.
- Avoid `optional.get()` without a check. Use `orElseThrow()`, `map`, `ifPresentOrElse`.

```
String city = userRepo.findById(id)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("UNKNOWN");
```

Q8. Explain JVM memory areas.

- **Heap**: all objects. It is shared by all threads and managed by the GC. It is split into the **Young generation** (Eden plus two Survivor spaces) and the **Old generation** (tenured).
- **Stack**: one per thread. Holds frames with local variables and partial results. `StackOverflowError` comes from deep recursion.
- **Metaspace** (Java 8+): class metadata in native memory. It replaced PermGen, and its size is limited with `-XX:MaxMetaspaceSize`.
- **PC register** and **native method stack** are per thread.
- **Code cache**: JIT-compiled native code.

> **Interview tip:** Common OutOfMemoryErrors: "Java heap space" (a leak or undersized heap), "Metaspace" (classloader leaks, often from repeated hot redeploys), "GC overhead limit exceeded", and "unable to create native thread".

Q9. How does Garbage Collection work? Compare G1, ZGC and Parallel GC.

Objects that can no longer be reached from **GC roots** (thread stacks, static fields, JNI refs) are collected. Most objects die young (the **generational hypothesis**), so the JVM collects the Young generation often (minor GC) and the Old generation rarely (major/full GC).

- **Serial**: single-threaded. For small heaps and client apps.
- **Parallel GC**: multi-threaded and throughput-focused. Pauses can be long. Good for batch jobs.
- **G1** (default since Java 9): the heap is divided into regions and the collector cleans the regions with the most garbage first. You set a pause target with `-XX:MaxGCPauseMillis=200`. A good general default for services.

- **ZGC / Shenandoah**: concurrent, low-latency collectors with sub-millisecond pauses even on multi-terabyte heaps. Generational ZGC is available from Java 21. Use them for latency-sensitive APIs.

> **Interview tip:** Tuning story for interviews: set `-Xms` equal to `-Xmx` in containers (or use `-XX:MaxRAMPercentage=75`), turn on GC logs with `-Xlog:gc*`, analyse them with GCeasy, take a heap dump on OOM with `-XX:+HeapDumpOnOutOfMemoryError`, and use Eclipse MAT to find leaks.

Q10. What is the Java Memory Model? Explain volatile, happens-before and visibility.

The JMM defines when a write made by one thread becomes **visible** to another thread. CPUs and the JIT reorder instructions and cache values in registers or CPU caches, so without synchronisation a thread may never see another thread's update.

- **volatile** guarantees **visibility** and **ordering**: reads and writes go to main memory and cannot be reordered around it. It does **not** give atomicity, so `count++` is still unsafe.
- **Happens-before rules**: unlocking a monitor happens-before a later lock of it; a volatile write happens-before later reads of it; `Thread.start()` happens-before actions in the started thread; actions in a thread happen-before another thread's `join()` returns.
- **synchronized** gives both **atomicity** (mutual exclusion) and **visibility**.

```
// Double-checked locking singleton needs volatile
public class Config {
    private static volatile Config instance;
    public static Config get() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) instance = new Config();
            }
        }
        return instance;
    }
}
```

Q11. synchronized vs ReentrantLock vs atomic classes?

- **synchronized**: simple and scoped to a block, and the lock is released automatically. You cannot use a timeout or interrupt a thread waiting for it.
- **ReentrantLock**: `tryLock(timeout)`, interruptible locking, fairness policy, and multiple `Condition`s. You **must** unlock in `finally`.
- **ReadWriteLock / StampedLock**: many readers or one writer. StampedLock also supports optimistic reads.
- **AtomicInteger / AtomicLong / AtomicReference**: lock-free CAS (compare-and-swap). The best choice for counters. For very high contention use **LongAdder**.
- **Java 21 note**: with virtual threads, long blocking inside `synchronized` could **pin** the carrier thread (fixed in JDK 24). ReentrantLock avoids this.

Q12. Explain the ExecutorService, thread pools and how you size them.

Creating threads is expensive, so pools reuse them. `ThreadPoolExecutor(core, max, keepAlive, queue, threadFactory, rejectionHandler)`.

- A task is submitted -> if fewer than core threads exist, a new thread starts -> otherwise the task is queued -> if the queue is full and there are fewer than max threads, a new thread starts -> otherwise the **RejectedExecutionHandler** runs (Abort, CallerRuns, Discard, DiscardOldest).

- **Avoid** `Executors.newFixedThreadPool` and `newCachedThreadPool` in production. The first uses an **unbounded queue**, which risks OOM. The second allows **unbounded threads**.
- **Sizing**: for CPU-bound work, threads = number of cores (+1). For I/O-bound work, threads = cores x (1 + wait time / compute time).
- Always shut down gracefully with `shutdown()`, `awaitTermination()` and then `shutdownNow()`.
- Name your threads with a custom ThreadFactory. It makes thread dumps much easier to read.

Q13. How does CompletableFuture work? Give a real example.

CompletableFuture lets you compose async stages without blocking.

- `supplyAsync` / `runAsync` start the work. Always pass your **own executor**, otherwise the common ForkJoinPool is used.
- `thenApply` (map), `thenCompose` (flatMap, for dependent async calls), `thenCombine` (joins two independent futures), `allOf` / `anyOf`.
- Error handling: `exceptionally`, `handle`, `whenComplete`. Timeouts: `orTimeout`, `completeOnTimeout` (Java 9+).

```
CompletableFuture<User> userF = supplyAsync(() -> userClient.get(id), pool);
CompletableFuture<List<Order>> ordersF = supplyAsync(() -> orderClient.get(id), pool);

Dashboard d = userF.thenCombine(ordersF, Dashboard::new)
    .orTimeout(2, TimeUnit.SECONDS)
    .exceptionally(ex -> Dashboard.fallback(id))
    .join();
```

> **Interview tip:** This cuts latency from t1 + t2 to max(t1, t2). It is a common way to aggregate data in a BFF or API-composition layer.

Q14. What are virtual threads and when would you use them?

Virtual threads (Java 21) are **lightweight threads managed by the JVM**, not the OS. Millions of them can exist. When a virtual thread blocks on I/O, the JVM **unmounts** it from its carrier (platform) thread, so the carrier is free to run other work.

- They are ideal for **thread-per-request, I/O-heavy** services. You can write simple blocking code and get throughput close to reactive code.
- They don't make CPU-bound code faster. Don't pool them. Be careful with `ThreadLocal`-heavy code, because there can be millions of copies.
- Spring Boot 3.2+: set `spring.threads.virtual.enabled=true`.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> executor.submit(() -> callApi(i)));
}
```

Q15. What causes deadlock and how do you detect and prevent it?

Deadlock happens when four conditions hold: mutual exclusion, hold-and-wait, no preemption and **circular wait**. For example, thread A holds lock 1 and wants lock 2, while thread B holds lock 2 and wants lock 1.

- **Prevent** it by always acquiring locks in a **global order**, using `tryLock` with a timeout, keeping lock scopes small and preferring higher-level concurrency utilities.
- **Detect** it with `jstack <pid>` or `jcmd <pid> Thread.print` (prints "Found one Java-level deadlock"), VisualVM or JMC, or `ThreadMXBean.findDeadlockedThreads()` in code.
- Related problems: **livelock** (threads keep reacting to each other and make no progress) and **starvation**.

Q16. Why must you override equals() and hashCode() together?

The contract says: if `a.equals(b)` then `a.hashCode() == b.hashCode()`. The reverse does not have to hold.

- Hash-based collections find the **bucket by hashCode** and then compare with **equals**. If you override only equals, two "equal" objects land in different buckets, so the HashSet keeps duplicates and `map.get()` fails.
- Use the **same immutable fields** in both methods. If a field used in hashCode changes after the object is inserted into a HashMap, the entry is effectively lost.
- Records generate correct equals and hashCode automatically.

Q17. How do you design an immutable class?

- Make the class `final` (or give it private constructors) so no one can subclass it.
- Make all fields `private final`.
- Provide no setters.
- **Defensively copy** mutable inputs and outputs, e.g. `List.copyOf(list)` or `new Date(d.getTime())`.
- Return new objects from "modifier" methods, e.g. `withSalary()`.
- Benefits: thread-safe without locks, safe to use as HashMap keys, safe to cache, easy to reason about. `String`, `Integer` and `LocalDate` are immutable.

```
public final class Money {
    private final BigDecimal amount; private final String currency;
    public Money(BigDecimal amount, String currency) {
        this.amount = Objects.requireNonNull(amount);
        this.currency = Objects.requireNonNull(currency);
    }
    public Money add(Money o) { return new Money(amount.add(o.amount), currency); }
}
// Or simply: public record Money(BigDecimal amount, String currency) {}
```

Q18. Why is String immutable? String pool, StringBuilder vs StringBuffer?

- **Security**: strings are used for class names, URLs, DB credentials and file paths.
- **String pool**: literals are interned and shared, which saves memory. This is only safe because strings are immutable.
- **Caching hashCode**: this makes String a fast HashMap key.
- **Thread safety**.
- `new String("a")` creates a new heap object. The literal `"a"` comes from the pool, so `"a" == "a"` is true while `new String("a") == "a"` is false. Always compare with `equals`.
- **StringBuilder**: mutable and not synchronized, so it's fast. **StringBuffer**: synchronized and legacy. Use StringBuilder when concatenating in loops.
- Java 9+ **compact strings** store Latin-1 text as one byte per character.

Q19. Checked vs unchecked exceptions? Best practices?

- **Checked** exceptions (subclasses of `Exception`, e.g. `IOException`) must be handled or declared. **Unchecked** exceptions (`RuntimeException`, e.g. `NullPointerException`, `IllegalArgumentException`) do not.
- `Error` (OOM, StackOverflow) should not be caught.
- **Best practices**: throw early and catch late. Never swallow exceptions. Wrap them and keep the cause (`new ServiceException("msg", e)`). Use try-with-resources for anything `AutoCloseable`. Build a custom exception hierarchy for the domain. In Spring, map exceptions

to HTTP responses centrally with `@ControllerAdvice`.

- Spring's `@Transactional` **rolls back only on unchecked exceptions by default**. Use `rollbackFor = Exception.class` if you also need rollback on checked ones.

Q20. Difference between abstract class and interface after Java 8?

- An **interface** can have default, static and private methods, but **no instance state** (only constants). A class can implement many interfaces. Use one to define a **capability or contract**.
- An **abstract class** can have state, constructors and protected members. A class can extend only one. Use it to share **common base behaviour and state**.
- **Diamond problem**: if two interfaces provide the same default method, the class must override it and choose, e.g. `A.super.method()`.

Q21. Explain ClassLoaders and how they delegate.

- **Bootstrap** (core JDK) -> **Platform/Extension** -> **Application/System** (the classpath) -> custom loaders.
- **Parent delegation**: a loader first asks its parent, and only loads the class itself if the parent cannot. This prevents core classes like `java.lang.String` from being spoofed.
- A class is identified by **(class name + classloader)**. The same class loaded by two loaders gives `ClassCastException`. This is common in app servers and during hot redeploys, and a leaked classloader causes Metaspace OOM.

Q22. What is serialization? What are its risks?

Serialization turns an object into bytes (`Serializable`). `transient` fields are skipped. `serialVersionUID` controls version compatibility.

- **Risk**: deserializing untrusted data can lead to **remote code execution** through gadget chains. Prefer JSON (Jackson) or Protobuf/Avro for data crossing a boundary, and use serialization filters (`ObjectInputFilter`) if Java serialization cannot be avoided.

Q23. How do you troubleshoot a slow or high-CPU Java service in production?

1. Check the metrics first: CPU, heap, GC pause time, thread count, and p99 latency in App Insights, Grafana or Prometheus.

2. **High CPU**: run `top -H -p <pid>` to find the hot thread, convert its id to hex, then find it in `jstack` output. Or use async-profiler or JFR to get flame graphs.

3. **Memory**: look at the GC logs, take a heap dump with `jcmd <pid> GC.heap_dump`, and use MAT's dominator tree.

4. **Latency**: use distributed tracing to find the slow downstream call. Check the DB slow query log, connection pool (HikariCP) waits and thread pool saturation.

5. Fix it, add a regression test or alert, and write a blameless postmortem.

> **Interview tip:** Interviewers at 9+ years expect a **structured method** backed by a real incident story.

2. Collections Framework (In Depth)

Q24. Explain the Collections Framework hierarchy.

- **Iterable** -> **Collection** -> **List** (ArrayList, LinkedList, Vector/Stack, CopyOnWriteArrayList)
- **Collection** -> **Set** (HashSet, LinkedHashSet, **SortedSet/NavigableSet**: TreeSet, plus EnumSet and CopyOnWriteArraySet)
- **Collection** -> **Queue** (PriorityQueue, **Deque**: ArrayDeque and LinkedList, **BlockingQueue**: ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue, DelayQueue, SynchronousQueue)
- **Map** is a **separate hierarchy** and does not extend Collection: HashMap, LinkedHashMap, TreeMap (SortedMap/NavigableMap), Hashtable, ConcurrentHashMap, WeakHashMap, IdentityHashMap, EnumMap, ConcurrentSkipListMap.
- Utility classes: `Collections` (sort, unmodifiableX, synchronizedX) and `Arrays`.
- Java 21 **SequencedCollection / SequencedMap** add `getFirst()`, `getLast()` and `reversed()` to ordered collections.

Q25. How does HashMap work internally? (The most-asked collections question)

- It is backed by an **array of buckets** (`Node<K,V>[] table`). The default capacity is **16** and the **load factor is 0.75**.
- **put(k, v)**: compute `hash = h ^ (h >>> 16)` (this spreads the high bits into the low bits), then `index = (n - 1) & hash`. Capacity is always a power of two, so this equals `hash % n` but is faster.
- If the bucket is empty, store a new node. Otherwise walk the bucket: if a key is `equals`, replace its value; if not, append to the chain.
- **Treeification (Java 8)**: when one bucket's chain grows beyond **8** nodes **and** the table capacity is at least **64**, the chain becomes a **red-black tree**. Worst-case lookup improves from O(n) to O(log n). It turns back into a list when it shrinks to **6**.
- **Resize**: when size > capacity x load factor (12 for the default), the table **doubles**. Every entry either stays at index `i` or moves to `i + oldCap`, decided by a single bit. Java 8 keeps the order, which fixed the infinite-loop problem Java 7 had during concurrent resize.
- One **null key** is allowed (it always goes to bucket 0), and many null values are allowed.
- **Not thread-safe**.

```
// Performance tip: pre-size when the count is known, to avoid rehashing
Map<String, User> map = new HashMap<>((int) (expected / 0.75f) + 1);
// Java 19+: HashMap.newHashMap(expected)
```

> **Interview tip:** Follow-up: "What if all keys return the same hashCode?" Everything goes into one bucket. Before Java 8 that meant O(n) lookups. From Java 8 it becomes a tree with O(log n) lookups, provided the keys are Comparable. This was the fix for **hash-flooding DoS** attacks.

Q26. HashMap vs Hashtable vs ConcurrentHashMap vs Collections.synchronizedMap?

- **HashMap**: not synchronized. Allows one null key and null values. The fastest option for a single thread.
- **Hashtable**: legacy. Every method is synchronized (one lock for the whole table). No nulls.

- **synchronizedMap**: a wrapper with one lock around every method. Iterating over it still needs a manual `synchronized(map)` block.
- **ConcurrentHashMap**: high concurrency. **No null keys or values**, because a null `get` would be ambiguous between "absent" and "mapped to null". Its iterators are weakly consistent.

Q27. How does ConcurrentHashMap achieve thread safety?

- **Java 7**: **Segments** (16 by default), each one a ReentrantLock guarding its own mini-table. This was lock striping.
- **Java 8+**: no segments. Inserting into an **empty bucket uses CAS** (no lock). A non-empty bucket is locked with `synchronized` **on the bucket's first node**, so only that bucket is locked. **Reads are lock-free** because node values and next pointers are volatile.
- **Resizing is cooperative**: threads that call `put` while a resize is running help move buckets (`ForwardingNode`).
- `size()` uses a counter-cell design like LongAdder. On big maps use `mappingCount()`.
- **Atomic compound operations**: `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`. Never write check-then-act code like `if (!map.containsKey(k)) map.put(k, v)`.

```
ConcurrentHashMap<String, LongAdder> hits = new ConcurrentHashMap<>();
hits.computeIfAbsent(url, k -> new LongAdder()).increment();

Map<String, Integer> wordCount = new ConcurrentHashMap<>();
words.forEach(w -> wordCount.merge(w, 1, Integer::sum));
```

Q28. ArrayList vs LinkedList? When would you use each?

- **ArrayList**: a dynamic array. `get(i)` is **O(1)**. Adding at the end is amortised O(1). Inserting or removing in the middle is O(n) because elements shift. Growth is 1.5x (`newCap = old + old >> 1`). It is **cache-friendly** because elements are contiguous.
- **LinkedList**: a doubly linked list. `get(i)` is O(n). Insert and remove are O(1) **only if you already hold the node** (through an iterator). Every node has extra memory overhead and poor cache locality.
- **In practice, ArrayList wins almost always**. For queue or stack use, prefer **ArrayDeque** to LinkedList.
- `Vector` is a synchronized, legacy version of ArrayList. `Stack` is legacy too, so use `ArrayDeque` instead.

Q29. HashSet, LinkedHashSet and TreeSet: how are they implemented?

- **HashSet** is backed by a **HashMap**: the element is the key and a dummy `PRESENT` object is the value. O(1) operations, no ordering.
- **LinkedHashSet** is backed by a LinkedHashMap and keeps **insertion order**.
- **TreeSet** is backed by a **TreeMap** (red-black tree). Elements are **sorted**, operations are O(log n), and it offers navigation methods: `floor`, `ceiling`, `headSet`, `tailSet`, `first`, `last`.
- TreeSet decides uniqueness with **compareTo/compare**, not `equals`. Keep compareTo consistent with equals, or elements will silently go missing.

Q30. How does LinkedHashMap work? Build an LRU cache.

LinkedHashMap is a HashMap plus a **doubly linked list** through its entries. It keeps **insertion order**, or **access order** when you pass `accessOrder=true`. Override `removeEldestEntry` and you have an LRU cache.

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LruCache(int capacity) {
```

```
        super(capacity, 0.75f, true);   // true = access order
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
// Not thread-safe: wrap it, or use Caffeine in production.
```

Q31. TreeMap internals and when to use it?

TreeMap is a **red-black tree**, a self-balancing BST. `get`, `put` and `remove` are O(log n). Keys are kept sorted by natural order or by a Comparator. No null keys (with natural ordering).

- **Use it for**: range queries (`subMap`, `headMap`, `tailMap`), nearest lookups (`floorKey`, `ceilingKey`), time-bucketed data, leaderboards and consistent-hashing rings.
- For concurrent use there is **ConcurrentSkipListMap**.

Q32. Comparable vs Comparator?

- **Comparable** (`compareTo`) is the **natural ordering** and lives inside the class. There is one per class.
- **Comparator** (`compare`) is an **external ordering strategy**. You can have many. Java 8 made them easy to compose:

```
employees.sort(Comparator.comparing(Employee::getDept)
    .thenComparing(Employee::getSalary, Comparator.reverseOrder())
    .thenComparing(Employee::getName, Comparator.nullsLast(String::compareTo)));
```

> **Interview tip:** Never compare by subtraction (`a - b`). It overflows. Use `Integer.compare(a, b)`.

Q33. Fail-fast vs fail-safe iterators?

- **Fail-fast** (ArrayList, HashMap): the collection has a `modCount`. If the collection is modified structurally during iteration (except through `iterator.remove()`), the next call throws **ConcurrentModificationException**. This is best-effort and happens even with a single thread.
- **Fail-safe / weakly consistent** (ConcurrentHashMap, CopyOnWriteArrayList): no exception. They iterate over a snapshot (CopyOnWrite) or a weakly consistent view (CHM), so they may not show recent changes.

```
// Wrong: throws ConcurrentModificationException
for (String s : list) if (s.isEmpty()) list.remove(s);
// Right
list.removeIf(String::isEmpty);
```

Q34. What is CopyOnWriteArrayList and when should you use it?

Every write (add or set) **copies the whole underlying array**. Reads need no locks and iterators use a snapshot.

- **Use it** when reads vastly outnumber writes, e.g. listener or observer lists and rarely changing configuration.
- **Avoid it** when writes are frequent or the list is large. Each write is O(n) and creates garbage.

Q35. Explain BlockingQueue implementations and the producer-consumer pattern.

- `put()` blocks when the queue is full and `take()` blocks when it is empty. `offer` and `poll` accept timeouts.
- **ArrayBlockingQueue**: bounded, array-backed, one lock.
- **LinkedBlockingQueue**: optionally bounded (unbounded by default, so be careful), with separate put and take locks for higher throughput.
- **PriorityBlockingQueue**: unbounded and ordered by priority.
- **DelayQueue**: elements become available only after their delay expires. Useful for scheduled retries.
- **SynchronousQueue**: no capacity, every put waits for a take. Used by cached thread pools.

```
BlockingQueue<Order> q = new ArrayBlockingQueue<>(1000);
// producer
q.put(order);
// consumer
while (running) { Order o = q.take(); process(o); }
```

Q36. PriorityQueue: how does it work? Solve "top K".

It is a **binary min-heap** stored in an array. `offer` and `poll` are O(log n), `peek` is O(1). Iteration order is **not** sorted.

```
// Top K largest numbers: keep a min-heap of size K -> O(n log k)
PriorityQueue<Integer> heap = new PriorityQueue<>();
for (int n : nums) {
    heap.offer(n);
    if (heap.size() > k) heap.poll();
}
```

Q37. WeakHashMap, IdentityHashMap and EnumMap: use cases?

- **WeakHashMap**: keys are held through **WeakReferences**. An entry disappears once its key is no longer strongly referenced anywhere else. Used for metadata caches keyed by objects with a lifecycle. Keys such as String literals never get collected.
- **IdentityHashMap**: compares keys with `==` instead of equals. Used for graph traversal, serialization and proxies.
- **EnumMap / EnumSet**: backed by an array or bit vector, so they are very fast and compact. Always prefer them when the keys are enums.

Q38. Unmodifiable vs immutable collections?

- `Collections.unmodifiableList(list)` is a **read-only view**. Changes to the underlying list still show through it.
- `List.of()`, `Set.of()`, `Map.of()` and `List.copyOf()` (Java 9/10) are **truly immutable**. They reject nulls, and `Set.of` / `Map.of` throw on duplicates. Iteration order of `Set.of` and `Map.of` is unspecified.
- `stream.toList()` (Java 16) returns an unmodifiable list. `Collectors.toList()` makes no such guarantee (it is an ArrayList in practice).

Q39. Big-O cheat sheet for common collections.

- **ArrayList**: get O(1), add O(1) amortised, insert/remove in the middle O(n), contains O(n).
- **LinkedList**: get O(n), add/remove at the ends O(1), contains O(n).
- **HashMap / HashSet**: get/put/remove O(1) on average, O(log n) worst case (tree bins, Java 8+).

- **TreeMap / TreeSet**: O(log n).
- **PriorityQueue**: offer/poll O(log n), peek O(1).
- **ArrayDeque**: add/remove at both ends O(1).

Q40. Can a mutable object be used as a HashMap key? What happens?

Technically yes, but it is dangerous. If you change a field used in hashCode after inserting the key, the entry stays in the **old bucket**. Both `get()` and `remove()` then fail and the entry leaks. **Always use immutable keys** (String, Integer, records, value objects).

3. Spring & Spring Boot

Q41. What is Spring Boot and how does auto-configuration work?

Spring Boot is an opinionated layer on top of Spring. It gives you **starters** (curated dependency sets), **auto-configuration**, an **embedded server** (Tomcat, Jetty or Undertow), **externalised configuration** and **Actuator**.

- `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.
- Auto-config classes are listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Boot 3; Boot 2 used `spring.factories`).
- Each one is guarded by **conditions**: `@ConditionalOnClass` (a library is on the classpath), `@ConditionalOnMissingBean` (you haven't defined your own), `@ConditionalOnProperty`, `@ConditionalOnWebApplication`.
- So if you define your own `DataSource` bean, Boot's auto-configured one **backs off**.
- Debug it with `--debug` or `debug=true`, which prints the condition evaluation report, or with Actuator's `/actuator/conditions`.

Q42. Explain IoC and Dependency Injection. Constructor vs field injection?

Inversion of Control: the container creates objects and wires their dependencies. Your classes don't use `new` for their collaborators.

- **Constructor injection (recommended)**: dependencies can be `final`, so the object is immutable. They are mandatory and visible. Classes are easy to unit test without Spring. Circular dependencies fail fast at startup. If a class has one constructor, `@Autowired` is optional.
- **Field injection** (`@Autowired` on a field): hides dependencies, cannot be final and makes testing harder. Avoid it.
- **Setter injection**: for optional dependencies.
- Ambiguous beans are resolved with `@Primary`, `@Qualifier("name")`, or by injecting `List<PaymentGateway>` / `Map<String, PaymentGateway>` (the strategy pattern).

Q43. Explain the Spring Bean lifecycle.

1. Instantiate (constructor).
2. Populate properties (DI).
3. Aware callbacks (`BeanNameAware`, `ApplicationContextAware`).
4. `BeanPostProcessor.postProcessBeforeInitialization`.
5. `@PostConstruct`, then `InitializingBean.afterPropertiesSet`, then a custom init-method.
6. `BeanPostProcessor.postProcessAfterInitialization`. **AOP proxies are created here.**
7. The bean is ready to use.
8. On shutdown: `@PreDestroy`, then `DisposableBean.destroy`, then a custom destroy-method.

Q44. Bean scopes? What happens when you inject a prototype into a singleton?

- **singleton** (default: one per container), **prototype** (new instance on every lookup), **request**, **session**, **application**, **websocket**.

- A prototype injected into a singleton is created **only once**, at injection time. To get a fresh instance each time, use `ObjectProvider<T>` / `Provider<T>`, `@Lookup` method injection, or a scoped proxy (`proxyMode = ScopedProxyMode.TARGET_CLASS`).
- Singleton beans are **not thread-safe by default**. Keep them stateless.

Q45. How does @Transactional work? What are the common pitfalls?

Spring wraps the bean in an **AOP proxy** (JDK dynamic proxy or CGLIB). The proxy opens a transaction, calls the method, then commits, or rolls back if an exception is thrown.

- **Pitfall 1: self-invocation.** Calling `this.otherTransactionalMethod()` from inside the same class **bypasses the proxy**, so no transaction starts. Fix it by moving the method to another bean or injecting the class's own proxy.
- **Pitfall 2**: only **public** methods are proxied (Spring 6 also allows protected and package-private with CGLIB). Private methods are ignored.
- **Pitfall 3**: rollback happens only for **RuntimeException and Error** by default. Use `rollbackFor = Exception.class`.
- **Pitfall 4**: if you catch the exception inside the method, the proxy never sees it, so nothing rolls back.
- **Pitfall 5**: long transactions that include remote calls hold DB connections and cause pool exhaustion.
- `readOnly = true` lets Hibernate skip dirty checking and lets the routing layer send the query to a read replica.

Q46. Explain transaction propagation and isolation levels.

Propagation:

- **REQUIRED** (default): join the existing transaction or create one.
- **REQUIRES_NEW**: suspend the current transaction and start a new independent one. Used for audit logs that must be saved even if the outer transaction fails.
- **NESTED**: savepoint inside the current transaction.
- **SUPPORTS**, **NOT_SUPPORTED**, **MANDATORY**, **NEVER**.

Isolation (what it prevents):

- **READ_UNCOMMITTED**: nothing (dirty reads possible).
- **READ_COMMITTED**: dirty reads. This is the default in PostgreSQL, Oracle and SQL Server.
- **REPEATABLE_READ**: dirty and non-repeatable reads. This is MySQL InnoDB's default.
- **SERIALIZABLE**: dirty reads, non-repeatable reads and phantoms. Safest but slowest.

Q47. What is the N+1 query problem in JPA/Hibernate and how do you fix it?

You load N parent entities with one query, and then accessing a **lazy** association runs **one more query per parent**, N in total.

- Fix with a **JOIN FETCH** in JPQL: `select o from Order o join fetch o.items`.
- Or an **@EntityGraph(attributePaths = "items")** on the repository method.
- Or `@BatchSize(size = 50)` / `hibernate.default_batch_fetch_size`, which loads the association with IN queries.
- Or **DTO projections** when you only need some fields.
- **Detect it** with `spring.jpa.show-sql`, Hibernate statistics, or tests that assert the query count.
- Also: keep associations **LAZY** (make `@ManyToOne` explicitly LAZY, since it defaults to EAGER) and turn off `spring.jpa.open-in-view` in APIs.

Q48. Optimistic vs pessimistic locking?

- **Optimistic**: add a `@Version` column. An update runs `WHERE id = ? AND version = ?`. If 0 rows change, you get `OptimisticLockException`, and you retry or return 409 to the caller. Best when conflicts are rare and reads are frequent.
- **Pessimistic**: `@Lock(PESSIMISTIC_WRITE)` issues `SELECT ... FOR UPDATE`. Rows stay locked until commit. Use it when there is high contention on critical rows (e.g. inventory or wallet balances), and watch for deadlocks.

Q49. How do you handle exceptions globally in a REST API?

Use `@RestControllerAdvice` with `@ExceptionHandler` methods. Return a consistent error body. Spring 6 supports RFC 7807 **ProblemDetail**.

```
@RestControllerAdvice
public class GlobalErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
            ex.getMessage());
        pd.setProperty("errorCode", "ORDER_404");
        return pd;
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException ex) { ... }
}
```

Q50. How does Spring AOP work? Where have you used it?

- Terms: **Aspect** (the module), **Advice** (the code: `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`), **Pointcut** (where it applies), **JoinPoint** (a method execution).
- Spring AOP is **proxy-based and applies at runtime**, to Spring beans and method calls only. AspectJ weaves at compile or load time and can intercept fields and constructors.
- Typical uses: logging, metrics and timing, auditing, security (`@PreAuthorize`), transactions, caching (`@Cacheable`), retry (`@Retryable`), and custom annotations such as `@Idempotent`.

```
@Around("@annotation(com.app.Timed)")
public Object time(ProceedingJoinPoint pjp) throws Throwable {
    long t = System.nanoTime();
    try { return pjp.proceed(); }
    finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - t) / 1_000_000);
        }
}
```

Q51. How do you manage configuration across environments?

- `application.yml` plus **profiles** (`application-dev.yml`, `application-prod.yml`), activated with `spring.profiles.active`.
- **Precedence** (highest first): command-line args > environment variables > profile-specific files > application.yml > defaults.
- Use type-safe `@ConfigurationProperties` records with validation instead of scattering `@Value` everywhere.
- **Secrets never go in git**. Load them from Azure Key Vault (the `spring-cloud-azure-starter-keyvault` property source), Kubernetes Secrets or environment variables.
- Centralised config: Spring Cloud Config Server or **Azure App Configuration**, with refresh via `@RefreshScope`.

Q52. What does Spring Boot Actuator give you?

Production endpoints: `/health` (with **liveness and readiness groups** for Kubernetes probes), `/metrics`, `/prometheus`, `/info`, `/env`, `/loggers` (change log levels at runtime), `/threaddump`, `/heapdump`.

- **Micrometer** is the metrics facade. It exports to Prometheus, Azure Monitor, Datadog and others.
- **Secure the endpoints**: expose only what is needed, and serve them on a separate management port.

Q53. How do you secure a Spring Boot microservice (JWT / OAuth2)?

- Use **Spring Security 6** with a `SecurityFilterChain` bean. `WebSecurityConfigurerAdapter` has been removed.
- The service is an **OAuth2 Resource Server**. It validates the JWT's signature (through JWKS), expiry, issuer and audience. Tokens are issued by an identity provider such as **Microsoft Entra ID (Azure AD)**, Keycloak or Okta.
- Authorisation: scopes and roles map to authorities, enforced with `@PreAuthorize("hasAuthority('SCOPE_orders.write')")`.
- Service-to-service calls use the **client credentials** flow, or managed identity on Azure.
- Also: HTTPS everywhere, CORS configuration, CSRF off only for stateless APIs, input validation, rate limiting at the gateway, and OWASP dependency scanning.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http.csrf(c -> c.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

Q54. How do you test Spring Boot applications?

- **Unit tests**: JUnit 5 + Mockito, with no Spring context. This is quick and should be the majority of tests.
- **Slice tests**: `@WebMvcTest` (controllers + MockMvc), `@DataJpaTest` (repositories), `@JsonTest`.
- **Integration tests**: `@SpringBootTest` with **Testcontainers** (real Postgres, Kafka, Redis in Docker). Prefer them to H2 so tests match production.
- **Contract tests**: Spring Cloud Contract or Pact between consumer and provider.
- **Stub HTTP dependencies** with WireMock. `@MockitoBean` (Boot 3.4+) replaces `@MockBean`.

Q55. How does Spring's REST client story look today (RestTemplate, WebClient, RestClient)?

- **RestTemplate**: blocking and in maintenance mode.
- **WebClient**: reactive and non-blocking (WebFlux). It can also be used in a blocking way.
- **RestClient** (Spring 6.1): a modern, fluent, synchronous client. Recommended for MVC apps, especially with virtual threads.
- **HTTP Interface** (`@HttpExchange`): declarative clients like Feign, built into Spring.
- Always configure **connect and read timeouts**, connection pooling, retries with backoff, and circuit breakers.

Q56. Spring MVC vs WebFlux? When would you choose reactive?

- **MVC**: thread-per-request, blocking, with the Servlet stack. Simple to write and debug. With virtual threads it now scales very well.
- **WebFlux**: event loop (Netty), non-blocking, Reactor (`Mono` / `Flux`). Good for streaming, many concurrent slow I/O calls, and backpressure.
- Reactive only pays off if the **entire chain is non-blocking** (R2DBC, reactive drivers). It costs complexity and makes debugging harder. Since Java 21, many teams choose MVC + virtual threads.

Q57. How do you implement caching in Spring Boot?

- `@EnableCaching` with `@Cacheable`, `@CachePut` and `@CacheEvict`.
- Providers: Caffeine (local, in-process) or **Redis** (distributed; Azure Cache for Redis).
- Watch out for: TTL and eviction, **cache stampede** (fix with `sync = true`, locks or early refresh), stale data, cache keys, and serialisation.
- Choose between **cache-aside**, write-through and write-behind strategies.

Q58. How do you improve Spring Boot startup time and memory?

- Use lazy initialisation where it fits, trim unused starters, and use **Class Data Sharing (CDS)** or **CRaC**.
- **GraalVM native image** (Spring AOT) starts in milliseconds and uses little memory. Good for serverless or Azure Functions. The trade-offs are long builds and reflection configuration.
- Use layered jars / Buildpacks for efficient Docker layers, and tune JVM memory flags for containers.

4. SQL & Relational Databases

Q59. What are ACID properties? Explain each with an example.

- **Atomicity**: all or nothing. A money transfer debits A **and** credits B, or neither happens. This is implemented with an undo log / rollback.
- **Consistency**: a transaction moves the DB from one valid state to another, respecting constraints (PK, FK, CHECK, UNIQUE).
- **Isolation**: concurrent transactions don't see each other's partial work. How strictly depends on the isolation level.
- **Durability**: once committed, data survives a crash. This is implemented with the **write-ahead log (WAL / redo log)**, which is flushed to disk before the commit is acknowledged.

Q60. Explain the types of JOINs.

- **INNER JOIN**: only rows that match in both tables.
- **LEFT (OUTER) JOIN**: all rows from the left table, with NULLs where the right side has no match.
- **RIGHT JOIN**: the mirror of LEFT. **FULL OUTER JOIN**: all rows from both sides.
- **CROSS JOIN**: the Cartesian product (every combination).
- **SELF JOIN**: a table joined to itself, e.g. an employee and their manager.
- **Anti-join** (rows with no match): `LEFT JOIN ... WHERE right.id IS NULL`, or `NOT EXISTS`.

```
-- Customers who never placed an order
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;

-- Employee with manager name (self join)
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.id;
```

Q61. WHERE vs HAVING? What is the logical order of SQL execution?

- **WHERE** filters **rows before grouping**. **HAVING** filters **groups after aggregation**. HAVING can use aggregates such as `COUNT(*) > 5`.
- **Logical order**: FROM / JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT / OFFSET.
- This explains why a column alias from SELECT usually **cannot** be used in WHERE, but **can** be used in ORDER BY.

```
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_sal
FROM employees
WHERE status = 'ACTIVE'
GROUP BY dept_id
HAVING COUNT(*) > 10
ORDER BY avg_sal DESC;
```

Q62. Find the Nth highest salary. (Classic question, three ways.)

```
-- 1. Window function (best: handles ties, easy to read)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 3;

-- 2. OFFSET / LIMIT (PostgreSQL, MySQL)
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2;

-- 3. Correlated subquery (portable, but slow on large tables)
SELECT DISTINCT salary FROM employees e1
WHERE 2 = (SELECT COUNT(DISTINCT salary) FROM employees e2 WHERE e2.salary > e1.salary);

-- Nth highest salary PER DEPARTMENT
SELECT * FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees e
) t WHERE rnk = 2;
```

Q63. Explain window functions. ROW_NUMBER vs RANK vs DENSE_RANK?

A window function computes a value **across a set of related rows without collapsing them** (unlike GROUP BY). Syntax: `func() OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`.

- For salaries 100, 100, 90: **ROW_NUMBER** gives 1, 2, 3 (always unique). **RANK** gives 1, 1, 3 (leaves gaps). **DENSE_RANK** gives 1, 1, 2 (no gaps).
- **LAG / LEAD**: the previous or next row's value (month-over-month change).
- **SUM / AVG OVER**: running totals and moving averages. Also **NTILE**, **FIRST_VALUE** and **LAST_VALUE**.

```
-- Running total and change from previous month
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month) AS running_total,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly_sales;
```

Q64. How do you find and delete duplicate rows?

```
-- Find duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Delete duplicates, keeping the lowest id (PostgreSQL / SQL Server style)
WITH ranked AS (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM users
)
DELETE FROM users WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
-- Then prevent it from happening again:
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);
```

Q65. What are CTEs and recursive CTEs?

A **CTE** (`WITH` clause) is a named temporary result set. It makes complex queries readable and can be referenced several times. A **recursive CTE** walks hierarchies: org charts, category trees, bill of materials.

```
-- All reports under manager 1, at any depth
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS level FROM employees WHERE id = 1
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.level + 1
    FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org;
```

Q66. Subquery vs JOIN vs EXISTS? IN vs EXISTS?

- A **correlated subquery** runs once per outer row, which can be slow. Modern optimisers often rewrite it as a join.
- **EXISTS** stops at the first match and is usually best for "does a related row exist?" checks.
- **NOT IN is a trap**: if the subquery returns any **NULL**, `NOT IN` returns **no rows**. Use `NOT EXISTS` instead.
- **UNION** removes duplicates (so it sorts or hashes). **UNION ALL** keeps them and is faster. Use UNION ALL when duplicates are impossible or acceptable.

Q67. DELETE vs TRUNCATE vs DROP?

- **DELETE**: DML. Removes rows (you can add WHERE), logs each row, fires triggers, and can be rolled back.
- **TRUNCATE**: DDL. Removes all rows quickly by deallocating pages. Minimal logging, resets identity counters, no row triggers. Can be rolled back in PostgreSQL and SQL Server inside a transaction, but not in MySQL or Oracle.
- **DROP**: removes the table itself (structure, data, indexes, constraints).

Q68. How do indexes work? Clustered vs non-clustered?

- Most indexes are **B+ trees**: balanced, sorted, with O(log n) lookup, range scans and ordering. Leaf pages are linked, which makes range scans fast.
- **Clustered index**: the table rows **are stored in index order**, so there is only one per table. In SQL Server and MySQL InnoDB it is usually the primary key.
- **Non-clustered (secondary) index**: a separate structure that stores key -> row pointer (or the PK in InnoDB). A lookup may need an extra **key lookup / bookmark lookup** to fetch the other columns.
- **Covering index**: contains every column the query needs (`INCLUDE` columns), so it avoids the lookup completely.
- Other types: **Hash** (equality only), **GIN / GiST** (PostgreSQL full-text, JSONB, geospatial), **bitmap** (Oracle, low cardinality), **partial / filtered** indexes (`WHERE status = 'ACTIVE'`).
- **The cost**: every index slows INSERT, UPDATE and DELETE and uses storage. Index for your real query patterns, not for every column.

Q69. What is a composite index and the leftmost prefix rule?

An index on `(last_name, first_name, city)` is sorted by last_name first, then first_name, then city.

- It helps queries filtering on `last_name`, on `last_name + first_name`, or on all three.
- It does **not** help a query that filters only on `first_name` or `city`, because the leftmost column is missing.
- **Column order**: put equality filters first and range filters (`>`, `BETWEEN`) last. Put more selective columns earlier when that fits the queries.

Q70. Why might a query NOT use an index?

- A function on the indexed column: `WHERE UPPER(email) = ...` or `WHERE YEAR(created_at) = 2025`. Fix it with a function-based index or rewrite the condition as a range.
- **Implicit type conversion**: comparing a VARCHAR column with a number.
- A leading wildcard: `LIKE '%abc'`. (`LIKE 'abc%'` can use the index.)
- Low selectivity: the optimiser chooses a full scan when most rows match anyway.

- `OR` across different columns, and stale **statistics** (run `ANALYZE` / update statistics).
- Missing the leftmost column of a composite index.

Q71. How do you optimise a slow SQL query?

1. Get the **execution plan**: `EXPLAIN ANALYZE` (PostgreSQL), `EXPLAIN` (MySQL), the actual execution plan (SQL Server).

2. Look for **full table scans**, bad row estimates, nested loops over large sets, sorts that spill to disk, and key lookups.

3. Add or adjust indexes (composite, covering), and make the conditions **sargable** (able to use an index).

4. Select only the columns you need (no `SELECT *`), and paginate with **keyset pagination** (`WHERE id > :lastId LIMIT 50`) instead of a large OFFSET.

5. Fix the N+1 pattern in the application, batch inserts, and keep transactions short.

6. Denormalise, add materialised views, partition large tables, or cache when it's justified.

Q72. Explain normalization (1NF, 2NF, 3NF, BCNF). When do you denormalize?

- **1NF**: atomic values, no repeating groups (no "phone1, phone2" columns and no comma-separated lists).
- **2NF**: 1NF, and every non-key column depends on the **whole** primary key (no partial dependency on part of a composite key).
- **3NF**: 2NF, and **no transitive dependency** (non-key columns depend only on the key). For example, don't store `dept_name` in employees when `dept_id` already determines it.
- **BCNF**: every determinant is a candidate key (a stricter 3NF).
- **Denormalize** for read-heavy workloads, reporting, and to avoid expensive joins: star schemas in warehouses, cached totals, read models in CQRS. The price is duplicated data that must be kept in sync.

Q73. Explain concurrency anomalies and isolation levels in the database.

- **Dirty read**: reading uncommitted data. **Non-repeatable read**: the same row read twice gives different values. **Phantom read**: the same query returns new rows. **Lost update**: two transactions overwrite each other's change.
- **READ COMMITTED** prevents dirty reads. **REPEATABLE READ** also prevents non-repeatable reads. **SERIALIZABLE** prevents everything.
- **MVCC** (PostgreSQL, Oracle, MySQL InnoDB): readers see a **snapshot** and don't block writers, and writers don't block readers. Old row versions are cleaned up later (PostgreSQL VACUUM).
- Prevent lost updates with `SELECT ... FOR UPDATE`, optimistic locking (a version column), or atomic updates: `UPDATE account SET balance = balance - 100 WHERE id = 1 AND balance >= 100`.

Q74. What causes database deadlocks and how do you handle them?

Two transactions lock rows in **opposite order** and each waits for the other. The DB detects the cycle and kills one of them (the victim).

- **Prevention**: access tables and rows in a consistent order, keep transactions short, add the right indexes (without them, updates lock more rows), and use lower isolation where it is safe.

- **Handling**: catch the deadlock error and **retry** the transaction with backoff. In Spring, use `@Retryable` on `DeadlockLoserDataAccessException` / `CannotAcquireLockException`.

Q75. What are views, materialized views, stored procedures and triggers?

- **View**: a saved query. It simplifies access and can restrict columns for security. Normal views store no data.
- **Materialized view**: stores the **result** physically and is refreshed on a schedule or on demand. Fast reads for reports, but the data can be stale.
- **Stored procedure**: logic that runs inside the DB. Fewer round trips, but harder to version, test and scale. In microservices, keep business logic in the application.
- **Trigger**: runs automatically on INSERT, UPDATE or DELETE (audit columns, history tables). Use it sparingly, because hidden side effects surprise developers.

Q76. What is database partitioning and how is it different from sharding?

- **Partitioning**: splits one large table into parts **inside the same DB server**, by **range** (dates), **list** (region) or **hash**. Queries touch only the relevant partitions (**partition pruning**). Old data can be archived quickly by dropping a partition.
- **Sharding**: spreads data across **multiple DB servers**, each holding a subset. It scales writes and storage, but makes cross-shard joins and transactions and rebalancing hard.
- Tools: Citus (PostgreSQL), Vitess (MySQL), Azure SQL Hyperscale, Cosmos DB's built-in partitioning.

Q77. How do you manage database schema changes in microservices?

- Use **Flyway** or **Liquibase**: versioned migration scripts checked into git and run at startup or in the pipeline.
- **Zero-downtime changes (expand and contract)**: add the new column (nullable) -> deploy code that writes to both columns -> backfill -> switch reads -> remove the old column in a later release.
- Never rename or drop a column in the same release that stops using it. Large table changes need online tools (gh-ost, pt-online-schema-change) or batched backfills.

Q78. Practical SQL questions to practise.

```
-- Department with the highest average salary
SELECT dept_id, AVG(salary) avg_sal FROM employees
GROUP BY dept_id ORDER BY avg_sal DESC LIMIT 1;

-- Employees earning more than their manager
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- Top 3 products by revenue in each category
SELECT * FROM (
  SELECT p.category, p.name, SUM(oi.qty * oi.price) revenue,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(oi.qty * oi.price) DESC) rn
  FROM order_items oi JOIN products p ON p.id = oi.product_id
  GROUP BY p.category, p.name
) t WHERE rn <= 3;

-- Users who logged in on 3 consecutive days
SELECT DISTINCT user_id FROM (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS grp
  FROM (SELECT DISTINCT user_id, login_date FROM logins) d
) t GROUP BY user_id, grp HAVING COUNT(*) >= 3;

-- Monthly active users
SELECT DATE_TRUNC('month', login_date) AS month, COUNT(DISTINCT user_id) AS mau
FROM logins GROUP BY 1 ORDER BY 1;
```

Q79. How do you size and tune a JDBC connection pool (HikariCP)?

- **More connections is not faster**. The DB can only run about (cores x 2 + effective spindles) queries in parallel. A small pool (10-20) per instance is typical.
- **Total connections = pool size x number of pods**. With 50 pods x 20 connections = 1,000 connections, which can exceed the DB's limit. Use PgBouncer, or reduce the pool size.
- Key settings: `maximumPoolSize`, `connectionTimeout` (fail fast), `maxLifetime` (shorter than the DB or firewall idle timeout), and `leakDetectionThreshold`.
- Watch the metrics: active and pending connections, and acquire time.

5. NoSQL Databases

Q80. What is NoSQL and what types exist?

NoSQL ("not only SQL") databases give up some relational features (joins, fixed schema, sometimes strong consistency) in exchange for **horizontal scale, flexible schemas and high performance** for specific access patterns.

- **Key-value**: Redis, DynamoDB, Azure Cache for Redis. Very fast lookups by key. Caches, sessions, rate limits, leaderboards.
- **Document**: MongoDB, Cosmos DB (NoSQL API), Couchbase. JSON documents with nested data, flexible schema. Catalogs, user profiles, content.
- **Wide-column**: Cassandra, HBase, ScyllaDB, Cosmos DB (Cassandra API). Massive write throughput. Time series, IoT, messaging, event logs.
- **Graph**: Neo4j, Cosmos DB (Gremlin API), Amazon Neptune. Relationships: social networks, fraud rings, recommendations.
- **Search**: Elasticsearch / OpenSearch, Azure AI Search. Full-text search, relevance ranking, log analytics.
- **Time-series**: InfluxDB, TimescaleDB, Azure Data Explorer. Metrics and sensor data.

Q81. When would you choose NoSQL over SQL?

- **Choose NoSQL** when you need huge scale or throughput beyond one server, a flexible or evolving schema, low-latency reads by key, global distribution, or data that is naturally shaped as aggregates (a document).
- **Choose SQL** when you need complex queries and joins, multi-row ACID transactions, strong integrity constraints, ad-hoc reporting, or when the data is highly relational (finance, orders, inventory).
- The key difference: in SQL you **model the data, then write queries**. In NoSQL you **list the queries first, then model the data to serve them** (query-driven design).

> **Interview tip:** Senior answer: "It is not either/or. We used PostgreSQL for orders and payments, Redis for caching and rate limits, Cosmos DB for the product catalog and Elasticsearch for search: polyglot persistence."

Q82. What is BASE and eventual consistency?

- **BASE**: **B**asically **A**vailable, **S**oft state, **E**ventual consistency. It is the counterpart of ACID for distributed stores that choose availability.
- **Eventual consistency**: if no new writes occur, all replicas **converge** to the same value eventually. Reads may briefly return stale data.
- Stronger options in between: **read-your-writes**, **monotonic reads**, **session consistency** (Cosmos DB's default), **bounded staleness**.
- Conflict resolution for concurrent writes: last-writer-wins (by timestamp), vector clocks, CRDTs, or custom merge logic.

Q83. How do you model data in a document database (MongoDB / Cosmos DB)?

- **Embed** data that is read together and owned by the parent (order + order lines, user + addresses). One read, atomic updates on a single document.

- **Reference** (store an ID) when the related data is large, unbounded (comments on a viral post), shared by many parents, or changes on its own.
- Avoid **unbounded arrays** and huge documents (MongoDB's limit is 16 MB, Cosmos DB's is 2 MB).
- Denormalize for reads (store `customerName` inside the order) and keep copies in sync with events or the change feed.
- Patterns: bucket (time series grouped by hour), subset (embed only the latest 10 reviews), computed (keep pre-aggregated totals), schema versioning (a `schemaVersion` field).

```
// Order document: lines embedded, customer referenced plus a denormalized name
{
  "id": "ord-1001", "customerId": "c-42", "customerName": "Asha",
  "status": "PAID", "total": 149.97,
  "lines": [ { "sku": "SKU-9", "qty": 3, "price": 49.99 } ],
  "createdAt": "2026-09-01T10:15:00Z"
}
```

Q84. How do you choose a partition key in Cosmos DB (or a shard key in general)?

- The partition key decides **how data is spread** across physical partitions and **which queries are cheap**.
- A good key has **high cardinality** (many distinct values), spreads **reads and writes evenly** (no hot partition), and appears in the **WHERE clause of most queries**, so they are single-partition queries.
- Bad keys: `status`, `country` or `date` (few values, or all of today's writes go to one partition). Common good keys: `tenantId`, `userId`, `deviceId`, or **hierarchical partition keys** (tenantId -> userId) for multi-tenant apps.
- Cross-partition queries fan out to every partition and cost more RU. In Cosmos DB, a logical partition holds at most 20 GB, and **the key cannot be changed** later without migrating the data.

Q85. Explain Cosmos DB key concepts: RU/s, consistency levels and change feed.

- **Request Units (RU/s)**: one currency for CPU, memory and I/O. A 1 KB point read by id plus partition key costs about **1 RU**. Queries and writes cost more. Provisioned (manual or autoscale) or serverless throughput. Exceeding it returns **HTTP 429**, and the SDK retries automatically.
- **Five consistency levels**: Strong, Bounded Staleness, **Session** (the default: read your own writes within a session), Consistent Prefix, Eventual. Stronger levels cost more RU and add latency.
- **Change feed**: an ordered log of changes per partition. Use it to build CQRS read models, sync to search, fire events (via Azure Functions), or run an outbox-like flow.
- **Multi-region writes**, automatic indexing (customise the indexing policy to cut RU cost), TTL per item, and APIs for NoSQL, MongoDB, Cassandra, Gremlin and Table.

Q86. How does Cassandra work? What is its data model?

- **Masterless ring**: every node is equal. Data is placed by **consistent hashing** of the **partition key**, and replicated N times (the replication factor).
- **Primary key = partition key + clustering columns**. The partition key picks the node, and clustering columns **sort rows inside the partition**.
- The write path is very fast: commit log -> memtable -> flushed to immutable **SSTables**. Compaction merges them. Deletes write **tombstones**.

- **Tunable consistency**: `ONE`, `QUORUM`, `ALL`. If **R + W > RF** (e.g. QUORUM reads and QUORUM writes with RF = 3), you get strong consistency.
- **Model one table per query**. There are no joins, so data is duplicated across tables on purpose.

```
-- Messages of a conversation, newest first
CREATE TABLE messages_by_conversation (
    conversation_id uuid,
    sent_at timeuuid,
    sender_id uuid,
    body text,
    PRIMARY KEY ((conversation_id), sent_at)
) WITH CLUSTERING ORDER BY (sent_at DESC);
```

Q87. What is Redis and what are its main use cases and data structures?

Redis is an **in-memory** key-value store with rich data structures. It is very fast (sub-millisecond) and runs commands on a single thread, so each command is **atomic**.

- **Strings**: cache values, counters (`INCR`), distributed locks (`SET key val NX PX 30000`).
- **Hashes**: objects and sessions. **Lists**: simple queues. **Sets**: unique items, tags.
- **Sorted sets (ZSET)**: leaderboards, priority queues, sliding-window rate limiting.
- **Streams**: a log with consumer groups (a light alternative to Kafka). **Pub/Sub**: fire-and-forget broadcast.
- **HyperLogLog** (approximate unique counts), **Bitmaps**, **Geo**.
- **Persistence**: RDB snapshots and/or AOF (append-only file). **Scaling**: replicas plus Sentinel for failover, or **Redis Cluster** (16,384 hash slots split across the primaries).
- **Eviction policies**: `allkeys-lru`, `volatile-ttl`, `noeviction`, and others. Always set a TTL on cache keys.

```
// Leaderboard with a sorted set
ZADD game:scores 1500 "asha"
ZINCRBY game:scores 50 "ravi"
ZREVRANGE game:scores 0 9 WITHSCORES   // top 10
```

Q88. How do you implement a distributed lock? What are the pitfalls?

- Redis: `SET lock:order:42 <uniqueToken> NX PX 30000` acquires the lock with an expiry. Release it with a **Lua script** that deletes the key only if the token matches, so you never delete someone else's lock.
- Libraries: **Redisson** (`RLock` with a watchdog that extends the lease), ShedLock (scheduled jobs in a cluster), or DB-based locks.
- **Pitfalls**: a GC pause or slow process can outlive the lease, so two holders exist at once. Use **fencing tokens** (a number that increases with every lock grant, checked by the storage layer) when correctness is critical. Redlock across several Redis nodes is debated. For strict guarantees, use ZooKeeper, etcd or a DB row lock.

Q89. How does Elasticsearch work and when would you use it?

- A distributed **search engine** built on Lucene. Documents are stored in an **inverted index** (term -> list of documents), which makes full-text search fast.
- An index is split into **primary shards** (fixed at creation) plus **replica shards**, spread across nodes.
- Text is **analyzed** (tokenised, lowercased, stemmed) using analyzers. Relevance scoring uses BM25.
- Use it for: product search with filters (facets), autocomplete, log analytics (ELK), geo search.

- **Not a primary database**: it is near-real-time (about a 1 s refresh), has no ACID transactions, and mapping changes need a reindex. Keep the source of truth in SQL/NoSQL and sync with CDC, events or the change feed.

Q90. How do you keep data consistent between a SQL database and a NoSQL / search store?

- **Avoid dual writes** from the application (one write can fail, leaving the stores inconsistent).
- Use **CDC** (Debezium reading the DB log) -> Kafka -> a sink connector or consumer that updates Elasticsearch or Redis.
- Or use the **Transactional Outbox** pattern, or the source store's change feed (Cosmos DB change feed -> Azure Function -> search index).
- Make the updates **idempotent** and version-aware (ignore older versions arriving late). Provide a **full reindex** job for recovery.

Q91. Rapid-fire SQL / NoSQL questions.

- **Primary key vs unique key?** A primary key is unique and NOT NULL, with only one per table (often clustered). A unique key allows NULLs (how many depends on the DB), and a table can have many.
- **What is a foreign key?** It enforces referential integrity between tables. Options: `ON DELETE CASCADE` / `SET NULL`. Index your FK columns.
- **CHAR vs VARCHAR?** CHAR is fixed-length and padded. VARCHAR is variable-length.
- **COUNT(*) vs COUNT(col)?** `COUNT(*)` counts all rows. `COUNT(col)` skips NULLs.
- **What is a surrogate key vs a natural key?** A generated ID (identity, UUID) vs a business value (email, PAN). Prefer surrogate keys, and put a unique constraint on the natural key.
- **OLTP vs OLAP?** Many small transactions (row store, normalised) vs large analytical queries (column store, star schema: Synapse, Fabric, Snowflake).
- **Does MongoDB support transactions?** Yes. Multi-document ACID transactions since 4.0 (replica sets) and 4.2 (sharded clusters). Single-document writes are always atomic.
- **What is a hot partition?** One partition receiving a disproportionate share of traffic, which throttles the whole workload. Fix it with a better key, a synthetic suffix on the key (write sharding), or caching.
- **Spring Data support?** Spring Data JPA (SQL), Spring Data MongoDB, Spring Data Redis, Spring Data Cassandra, Spring Data Elasticsearch, and Spring Cloud Azure's Cosmos DB starter all share the same repository abstraction.

6. Microservices Architecture & Patterns

Q92. Monolith vs microservices: when would you NOT use microservices?

Microservices are **independently deployable services, each organised around a business capability, each owning its own data**.

- **Benefits**: independent deployment and scaling, team autonomy (fits Conway's law), fault isolation, and freedom of technology per service.
- **Costs**: distributed-systems complexity (network failures, latency, eventual consistency), operational overhead (CI/CD, observability, Kubernetes), harder testing and debugging, and data consistency across services.
- **Avoid them** for a small team, an unclear domain or an early-stage product. Start with a **modular monolith** with clean boundaries, and extract services when a real need appears (scale, release cadence, team growth).

> **Interview tip:** A 9+ year answer shows judgement: "Microservices are an organisational scaling tool first and a technical one second."

Q93. How do you decide service boundaries? (Domain-Driven Design)

- Use **DDD bounded contexts**. Each context has its own ubiquitous language and model. For example "Product" means different things in Catalog, Inventory and Shipping.
- Find the contexts through **Event Storming** with domain experts, looking at business capabilities and subdomains (core, supporting, generic).
- **Aggregates** are consistency boundaries. One transaction should change one aggregate.
- **Signs of bad boundaries**: chatty synchronous calls between services, services that must always be deployed together (a distributed monolith), and shared databases.

Q94. Explain the API Gateway pattern.

One entry point for clients. It handles **routing, authentication (JWT validation), rate limiting, SSL termination, request aggregation, CORS, caching and API versioning**.

- Options: Spring Cloud Gateway, **Azure API Management**, Kong, NGINX, Azure Application Gateway (L7 load balancer + WAF).
- **BFF (Backend for Frontend)**: a separate gateway per client type (web, mobile), shaped to what each needs.
- Keep **business logic out** of the gateway.

Q95. Service discovery: client-side vs server-side?

- **Client-side**: the client asks a registry (Eureka, Consul) and load-balances itself (Spring Cloud LoadBalancer).
- **Server-side**: the client calls a stable name and the platform routes it. In **Kubernetes**, a Service's DNS name (`orders-svc.namespace.svc.cluster.local`) goes through kube-proxy to the pods. On Kubernetes you usually **don't need Eureka**.

Q96. Synchronous vs asynchronous communication between services?

- **Synchronous** (REST, gRPC): simple request/response with immediate consistency. But it creates **temporal coupling** (both services must be up) and latency adds up along the chain.
- **Asynchronous** (Kafka, Azure Service Bus, RabbitMQ): loose coupling, buffering and resilience, and easy fan-out to many consumers. The costs are eventual consistency, harder tracing, and the need for idempotency and message ordering.
- **Rule of thumb**: queries that need an answer now are synchronous. **Commands and state-change notifications are asynchronous** through events. gRPC is a good fit for high-throughput internal calls.

Q97. Explain the Circuit Breaker pattern (Resilience4j).

It stops calls to a failing dependency so failures don't cascade and exhaust threads.

- **CLOSED**: calls pass through and failures are counted over a sliding window.
- **OPEN**: once the failure rate exceeds a threshold (e.g. 50%), calls **fail fast** or go to a fallback, for a wait period.
- **HALF_OPEN**: a few trial calls are allowed. If they succeed the breaker goes back to CLOSED, otherwise back to OPEN.
- Combine it with **timeouts, retries with exponential backoff and jitter, bulkheads** (isolated thread pools or semaphores per dependency) and **rate limiters**.

```
@CircuitBreaker(name = "inventory", fallbackMethod = "stockFallback")
@Retry(name = "inventory")
@TimeLimiter(name = "inventory")
public CompletableFuture<Stock> getStock(String sku) { ... }

resilience4j.circuitbreaker.instances.inventory:
  slidingWindowSize: 20
  failureRateThreshold: 50
  waitDurationInOpenState: 30s
```

> **Interview tip:** Retry only **idempotent** operations. Retry storms can take down a service that is trying to recover.

Q98. How do you handle distributed transactions? Explain the Saga pattern.

2PC/XA does not scale and does not fit microservices, where each service has its own DB, often a NoSQL store or a broker. A **Saga** is a sequence of **local transactions**. Each step publishes an event or command. If a step fails, **compensating transactions** undo the earlier steps.

- **Choreography**: services react to each other's events. There is no central coordinator. It works for simple flows, but becomes hard to follow as the flow grows.
- **Orchestration**: a central orchestrator (a state machine, Temporal, Camunda or Azure Durable Functions) tells each service what to do. The flow is explicit and easier to monitor.
- Example (orders): `OrderCreated` -> PaymentService charges -> `PaymentCompleted` -> InventoryService reserves stock -> if that fails, `InventoryFailed` -> **refund the payment** -> cancel the order.
- Sagas give no isolation, so handle anomalies with **semantic locks** (e.g. status PENDING), commutative updates and re-reading values.

Q99. What is the Transactional Outbox pattern and why do you need it?

The dual-write problem: saving to the DB and publishing to Kafka are two separate systems. If one succeeds and the other fails, they become inconsistent.

- **Outbox**: in the **same local DB transaction**, write the business row **and** an `outbox` row holding the event.
- A separate relay publishes the outbox rows to Kafka. Either a **poller**, or better, **CDC with Debezium** reading the DB transaction log.
- This gives **at-least-once** delivery, so consumers must be **idempotent**.
- The mirror image is the **Inbox pattern**: consumers store processed message IDs to deduplicate.

Q100. Explain CQRS and Event Sourcing.

- **CQRS**: separate the **write model** (commands, a normalised and validated domain) from the **read model** (queries, denormalised views such as Elasticsearch, Cosmos DB or a Redis projection). The two can scale separately. The read model is updated from events and is eventually consistent.
- **Event Sourcing**: store **every state change as an immutable event** instead of the current state. State is rebuilt by replaying events (use snapshots for speed). You get a full audit trail, time travel and replay into new projections.
- **Costs**: complexity, schema evolution of events, and eventual consistency. Use them for audit-heavy domains (banking, ledgers, orders), not for simple CRUD.

Q101. How do you make APIs and consumers idempotent?

Retries and at-least-once delivery mean the **same request or message can arrive more than once**.

- The client sends an **Idempotency-Key** header (like Stripe). The server stores the key and its response, and returns the stored response for a repeated key.
- Consumers keep a `processed_messages` table (message ID with a unique constraint) or make **natural idempotent updates** (`SET status = 'PAID'` rather than `balance = balance - x`).
- Use DB unique constraints and upserts. PUT and DELETE are idempotent by definition. POST is not, unless you add a key.

Q102. Database per service: how do you run queries that span services?

- Each service **owns its data**. No other service reads its tables directly.
- For cross-service queries: **API composition** (an aggregator calls several services), a **CQRS read model** built from events, or **data replication through events** (keep a local copy of the data you need).
- Reporting and analytics: stream events into a data lake or warehouse (e.g. Azure Synapse or Fabric).

Q103. How do you implement observability in microservices?

The **three pillars**:

- **Logs**: structured JSON logs with a **correlation / trace ID** in every line (MDC). Centralise them in ELK or Azure Log Analytics.
- **Metrics**: RED (Rate, Errors, Duration) and USE (Utilisation, Saturation, Errors). Micrometer -> Prometheus -> Grafana, or Azure Monitor.
- **Traces**: distributed tracing with **OpenTelemetry** (W3C traceparent header). In Spring Boot 3 this is Micrometer Tracing. Export to Jaeger, Zipkin or **Application Insights**.
- Also: health checks, SLOs and SLIs with error budgets, and alerts on symptoms (latency, error rate) rather than causes.

Q104. How do you version APIs and avoid breaking consumers?

- URI versioning (`/v1/orders`), header versioning or media-type versioning. URI versioning is the most practical.
- Prefer **backward-compatible changes**: add optional fields, never rename or remove fields, follow the tolerant-reader principle.
- Use **consumer-driven contract tests** (Pact) in CI, and announce deprecations with sunset headers.
- For events, use a **Schema Registry** with compatibility rules (BACKWARD or FULL).

Q105. What is the Strangler Fig pattern? How would you migrate a monolith?

Gradually replace pieces of the monolith. Put a **facade or gateway** in front of it, route one capability at a time to the new service, and retire the old code.

1. Identify seams and bounded contexts. Start with a low-risk, high-value capability.

2. Add an anti-corruption layer that translates between the old and new models.

3. Split the data: sync with CDC or events, then move ownership to the new service.

4. Run in parallel and compare (dark launches, feature flags), then cut over and delete the old code.

Q106. What other microservice patterns should you know?

- **Sidecar / Service Mesh** (Istio, Linkerd, Dapr): mTLS, retries, traffic shifting and telemetry without changing application code.
- **Bulkhead**: isolate resources per dependency so one failure doesn't drain everything.
- **Anti-Corruption Layer**: protect your domain model from legacy or external models.
- **Backends for Frontends**, **Aggregator**, **Ambassador**.
- **Health Check API**, **Externalised Configuration**, **Log Aggregation**.
- **Competing Consumers**, **Claim Check** (store a large payload in blob storage and send only a reference through the broker).

Q107. How do you secure service-to-service communication?

- **mTLS** through a service mesh, or with certificates.
- **OAuth2 client credentials**, or **Azure Managed Identity**, with no secrets in code. Pass the user context along as a JWT (token exchange / on-behalf-of flow).
- Network policies (Kubernetes NetworkPolicy), private endpoints, and zero-trust principles.
- Secrets in Key Vault, rotated automatically.

7. Design Principles & Design Patterns

Q108. Explain the SOLID principles with examples.

- **S - Single Responsibility**: a class should have one reason to change. `InvoiceService` should not also format PDFs and send emails.
- **O - Open/Closed**: open for extension, closed for modification. Add a new `PaymentStrategy` class instead of adding another `if/else` to existing code.
- **L - Liskov Substitution**: subtypes must be usable wherever the base type is expected. The classic violation is `Square extends Rectangle`, because setting the width silently changes the height.
- **I - Interface Segregation**: many small, client-specific interfaces are better than one fat interface. Clients should not depend on methods they don't use.
- **D - Dependency Inversion**: depend on abstractions, not concretions. Services depend on a `PaymentGateway` interface, and Spring injects the implementation.
- Related principles: **DRY**, **KISS**, **YAGNI**, **composition over inheritance**, **Law of Demeter**.

Q109. Singleton: how do you implement it correctly? What are its problems?

```
// 1. Enum singleton: thread-safe, and safe against serialization and reflection
public enum AppRegistry { INSTANCE; public void register() { ... } }

// 2. Initialization-on-demand holder: lazy and thread-safe, with no locking
public class Cache {
    private Cache() {}
    private static class Holder { static final Cache INSTANCE = new Cache(); }
    public static Cache getInstance() { return Holder.INSTANCE; }
}
```

- It can be broken by **reflection** (guard in the constructor), **serialization** (add `readResolve`), cloning, and multiple classloaders.
- **Problems**: global state, hidden dependencies, and difficult testing. In Spring, prefer **singleton-scoped beans** injected through DI.

Q110. Factory Method vs Abstract Factory?

- **Factory Method**: a method decides which concrete class to create. The creation logic lives in one place.
- **Abstract Factory**: creates **families of related objects** that belong together (e.g. `AzureCloudFactory` creates an AzureStorage and an AzureQueue, while `AwsCloudFactory` creates the AWS versions).
- In Spring, `BeanFactory` / `FactoryBean` and `@Bean` methods are factories.

```
@Component
class NotificationFactory {
    private final Map<String, NotificationSender> senders; // Spring injects all beans, keyed by
        bean name
    NotificationFactory(Map<String, NotificationSender> senders) { this.senders = senders; }
    NotificationSender get(Channel c) {
        return Optional.ofNullable(senders.get(c.beanName()))
            .orElseThrow(() -> new IllegalArgumentException("Unknown channel " + c));
    }
}
```

Q111. Builder pattern: when and why?

Use it for objects with **many optional parameters**. It avoids telescoping constructors, makes the calling code readable, and lets you build **immutable** objects with validation in `build()`.

- Examples: `StringBuilder`, `HttpRequest.newBuilder()`, `UriComponentsBuilder`, Lombok's `@Builder`, `WebClient.builder()`.

```
Order order = Order.builder()
    .customerId("C1").item("SKU-9", 2)
    .shippingAddress(addr).priority(Priority.HIGH)
    .build();   // build() validates the required fields
```

Q112. Strategy pattern: give a real-world example.

Define a family of interchangeable algorithms behind one interface and choose one at runtime. It **replaces if/else or switch chains** and follows the Open/Closed principle.

```
interface DiscountStrategy { BigDecimal apply(Order o); }
@Component("FESTIVE") class FestiveDiscount implements DiscountStrategy { ... }
@Component("LOYALTY") class LoyaltyDiscount implements DiscountStrategy { ... }

@Service
class PricingService {
    private final Map<String, DiscountStrategy> strategies;
    PricingService(Map<String, DiscountStrategy> strategies) { this.strategies = strategies; }
    BigDecimal price(Order o) { return strategies.get(o.getPromoType()).apply(o); }
}
```

- Examples in the JDK and Spring: `Comparator`, `ResourceLoader`, `PasswordEncoder` implementations, and payment or shipping providers.

Q113. Observer pattern and how it relates to event-driven systems?

Subjects notify their subscribers when state changes, which gives loose coupling.

- In Java/Spring: `ApplicationEventPublisher` + `@EventListener`, and `@TransactionalEventListener(phase = AFTER_COMMIT)` so the event is published only after the transaction commits.
- At system scale it becomes **pub/sub** over Kafka or Service Bus.
- Reactive Streams (`Publisher` / `Subscriber`) are the observer pattern with backpressure.

Q114. Decorator vs Proxy vs Adapter?

- **Decorator**: wraps an object with the **same interface** to **add behaviour**, and decorators can be stacked. Example: `BufferedInputStream(new FileInputStream(f))`. Or a caching or logging decorator around a repository.
- **Proxy**: same interface, but it **controls access**: lazy loading, security, remote calls, transactions. Spring AOP, `@Transactional` and Hibernate lazy entities are proxies.
- **Adapter**: **converts one interface into another** that the client expects. Examples: `Arrays.asList`, `InputStreamReader`, or wrapping a legacy SOAP client behind a clean domain interface (hexagonal architecture ports and adapters).

Q115. Template Method vs Strategy?

- **Template Method**: an abstract class defines the **skeleton of an algorithm**, and subclasses override specific steps. It is based on **inheritance**. Examples: `JdbcTemplate`, `AbstractController`, and batch jobs with read, process and write steps.
- **Strategy**: the whole algorithm is swapped through **composition**, which is more flexible.

Q116. Chain of Responsibility: where is it used?

A request passes along a chain of handlers. Each handler either handles it or passes it on.

- Examples: **Servlet Filters**, the **Spring Security filter chain**, `HandlerInterceptor`s, and validation pipelines (fraud check -> KYC check -> limit check).

Q117. Command, State and Template patterns in business workflows?

- **Command**: wraps a request as an object, which enables queuing, undo, retry and audit. Examples: `Runnable`, CQRS commands, and messages on a queue.
- **State**: an object changes behaviour when its internal state changes (an order moving through CREATED -> PAID -> SHIPPED -> DELIVERED). It replaces status switch statements. Spring State Machine is one implementation.

Q118. Which design patterns does the Spring Framework use?

- **Singleton** (bean scope), **Factory** (BeanFactory, FactoryBean), **Proxy** (AOP, @Transactional), **Template Method** (JdbcTemplate, RestTemplate, KafkaTemplate), **Observer** (ApplicationEvent), **Strategy** (resource loading, and the many pluggable interfaces), **Adapter** (HandlerAdapter in MVC), **Front Controller** (DispatcherServlet), **Decorator** (BeanPostProcessor wrapping), **Chain of Responsibility** (filters), **Dependency Injection / IoC**.

Q119. What are anti-patterns you have seen and fixed?

- **God class / god service**: split it by responsibility.
- **Anemic domain model**: move behaviour into the entities or aggregates when that makes sense.
- **Distributed monolith**: services that must be deployed together, or that share a DB.
- **Chatty services**: batch calls, or replicate data through events.
- **Shotgun surgery**, **copy-paste code**, **magic numbers**, **catching generic Exception and swallowing it**.
- **Premature optimisation** and **over-engineering** (a pattern where a simple function would do).

Q120. Explain Hexagonal / Clean Architecture.

- The **domain core** holds entities and use cases, and has **no framework dependencies**.
- **Ports** are interfaces defined by the core, both inbound (use cases) and outbound (repositories, gateways).
- **Adapters** implement the ports: REST controllers and Kafka listeners (inbound), JPA repositories and HTTP clients (outbound).
- **Dependencies point inward**. The payoffs are testability, swappable infrastructure, and business logic isolated from framework churn.

8. System Design

Q121. How do you approach a system design interview? (Framework)

1. **Clarify requirements** (5 min). Functional: what does the system do? Non-functional: scale, latency, availability, consistency, durability, security, cost.

2. **Estimate** (back-of-the-envelope): DAU, QPS (average and peak), storage per year, bandwidth, read/write ratio.

3. **API design**: the key endpoints or events.

4. **Data model** and database choice (SQL vs NoSQL, and why).

5. **High-level design**: clients -> CDN -> load balancer / gateway -> services -> cache -> DB -> queues -> workers.

6. **Deep dive** into the bottlenecks: sharding, caching, hot keys, consistency, failure handling.

7. **Trade-offs, monitoring, security** and how the design evolves.

> **Interview tip:** Talk through your trade-offs out loud. Interviewers score your reasoning, not a "correct" diagram.

Q122. Explain the CAP theorem and PACELC.

- **CAP**: during a **network Partition**, a distributed system must choose between **Consistency** (every read sees the latest write) and **Availability** (every request gets a response). Partitions are unavoidable, so the real choice is **CP or AP**.
- **CP** examples: ZooKeeper, etcd, HBase, and banking ledgers. **AP** examples: Cassandra, DynamoDB (by default), Cosmos DB with eventual consistency, and shopping carts.
- **PACELC**: if Partitioned, choose A or C; **E**lse (normal operation), choose between **L**atency and **C**onsistency.
- **Cosmos DB** offers five consistency levels: Strong, Bounded Staleness, **Session** (the default), Consistent Prefix and Eventual.

Q123. SQL vs NoSQL: how do you choose?

- **SQL** (PostgreSQL, Azure SQL): ACID, joins, strong consistency, complex queries, a stable schema. Use it for financial data, orders and anything relational.
- **NoSQL**:
- **Key-value** (Redis): caches and sessions.
- **Document** (MongoDB, Cosmos DB): flexible schema, aggregate-shaped data.
- **Wide-column** (Cassandra): huge write throughput and time-series data.
- **Graph** (Neo4j): relationships and recommendations.
- **Search** (Elasticsearch): full text search.
- Scaling: SQL is usually scaled up plus read replicas, then sharded. NoSQL is built to scale out.
- Many systems are **polyglot**: pick the store per use case.

Q124. How do you scale a database?

- **Indexing** and query optimisation first (check `EXPLAIN` plans, covering indexes, avoid `SELECT *`).
- **Read replicas** for heavy read traffic. Watch replication lag, and route read-your-own-writes queries to the primary.
- **Caching** (Redis) in front of the DB.

- **Vertical partitioning** (split tables or columns) and **sharding** (horizontal partitioning by shard key: hash, range or directory based).
- Choosing a **shard key**: high cardinality, even distribution, and it matches your query pattern. Avoid monotonically increasing keys with range sharding, because they create hot spots.
- **Consistent hashing** reduces data movement when nodes are added or removed (virtual nodes smooth the distribution).
- **Connection pooling** (HikariCP sizing, PgBouncer).

Q125. Explain caching strategies and cache invalidation.

- **Cache-aside (lazy loading)**: the app reads the cache, and on a miss reads the DB and fills the cache. This is the most common strategy.
- **Read-through / Write-through**: the cache sits in front of the DB and writes go to both. Consistent, but writes are slower.
- **Write-behind**: writes go to the cache and are flushed to the DB asynchronously. Fast, but data can be lost.
- **Eviction**: LRU, LFU, TTL.
- **Problems and fixes**:
- **Cache stampede** (many misses at once): locking or single-flight, request coalescing, early probabilistic refresh.
- **Cache penetration** (queries for keys that don't exist): cache null results, or use a Bloom filter.
- **Hot keys**: local L1 caches, key replication.
- **Invalidation**: short TTLs plus event-driven eviction (a DB change event evicts the key).
- Layers: browser -> **CDN** (Azure Front Door) -> gateway -> app (Caffeine) -> distributed (Redis) -> DB.

Q126. Design a URL shortener (like bit.ly).

- **Requirements**: 100M new URLs per month, a read:write ratio of 100:1, low-latency redirects, custom aliases, expiry, analytics.
- **Estimates**: about 40 writes/sec and about 4,000 reads/sec (plan for 10x peak). About 6B URLs over 5 years x 500 bytes = about 3 TB.
- **Key generation**: **Base62** of a unique ID (7 characters = 62^7, about 3.5 trillion). Get the IDs from a **distributed ID generator** (Snowflake, or pre-allocated ranges from a counter service in ZooKeeper or a DB) so there are no collisions. Hashing (MD5) plus collision checks also works.
- **Storage**: a key-value / NoSQL store (Cosmos DB, Cassandra, DynamoDB) keyed by the short code.
- **Read path**: CDN/edge -> Redis cache (the hot 20% of URLs) -> DB. Return **301** (cacheable, fewer hits to your servers) or **302** (so every click can be tracked).
- **Analytics**: publish click events to **Kafka** asynchronously, then stream-process them into an OLAP store.
- **Other points**: rate limiting, abuse / malware URL checks, and a TTL cleanup job.

Q127. Design a rate limiter.

- **Algorithms**:
- **Token Bucket**: tokens refill at rate r and the bucket holds up to b. Allows bursts. This is the most common.
- **Leaky Bucket**: a smooth, constant outflow.
- **Fixed Window Counter**: simple, but allows bursts at window edges.

- **Sliding Window Log**: accurate, but uses a lot of memory.
- **Sliding Window Counter**: a good approximation of the log.
- **Where**: at the API gateway (APIM `rate-limit-by-key`, Spring Cloud Gateway's RedisRateLimiter) or in a service.
- **Distributed**: keep the counters in **Redis** with atomic **Lua scripts** or `INCR` + `EXPIRE`, keyed by user, API key or IP.
- **Response**: HTTP **429** with `Retry-After` and `X-RateLimit-Remaining` headers.
- **Failure mode**: decide whether to fail open or fail closed if Redis is down.

Q128. Design a notification system (email / SMS / push).

- Services publish `NotificationRequested` events to **Kafka** or Service Bus.
- The **Notification service** validates the request, checks user preferences and opt-outs, renders templates, and deduplicates with an idempotency key.
- There is **one queue or topic per channel** (email, SMS, push). Workers call providers (SendGrid, Twilio, FCM/APNs, Azure Communication Services).
- **Retries with exponential backoff**, a **DLQ** for poison messages, and provider failover.
- **Rate limit** per user and per provider. Scheduling and quiet hours.
- Track delivery status (sent, delivered, opened) in a store, and expose it through an API.

Q129. Design an e-commerce order and payment system.

- Services: Catalog, Cart, **Order**, **Payment**, **Inventory**, Shipping, Notification.
- **Checkout flow** as a **Saga**: Create Order (PENDING) -> Reserve Inventory -> Charge Payment (idempotency key with the payment provider) -> Confirm Order -> Ship. Compensations: release the stock, refund the payment.
- Use the **Outbox** pattern for reliable events, and make every consumer idempotent.
- **Inventory contention** (flash sales): reserve with a TTL. Use atomic decrements in Redis, then sync to the DB. Put a queue in front to smooth the traffic spike.
- **Read side**: order history through CQRS projections, search through Elasticsearch.
- Security: PCI-DSS means you **never store card data** (use tokenisation).

Q130. Design a chat / messaging system (like WhatsApp).

- Persistent **WebSocket** connections to chat servers. A **connection registry** in Redis maps each user to a server.
- Message flow: sender -> chat server -> persisted to a store (Cassandra, partitioned by conversation ID and ordered by time) -> routed to the recipient's server -> delivered. If the recipient is offline, send a push notification and deliver on reconnect.
- **Ordering**: a per-conversation sequence number. **Delivery receipts**: sent, delivered, read.
- **Group chat**: fan-out on write for small groups and fan-out on read for very large ones.
- **Presence**: heartbeats with a TTL in Redis.
- **End-to-end encryption** (Signal protocol) if required.

Q131. How do you design for high availability and disaster recovery?

- **Remove single points of failure**: multiple instances behind a load balancer, spread across **Availability Zones**.
- **Multi-region**: active-passive (cheaper, RTO in minutes) or active-active (complex, conflict resolution). Use Azure Front Door or Traffic Manager for global routing.

- Define **RTO** (recovery time objective) and **RPO** (recovery point objective), and choose the replication mode to match (async vs sync, geo-redundant storage).
- **Test it**: backups with restore drills, chaos engineering (Azure Chaos Studio), runbooks.
- **Graceful degradation**: feature flags, fallbacks, serving stale cache data.

Q132. How do you generate unique IDs in a distributed system?

- **UUID v4**: random, with no coordination needed, but 128 bits long and it fragments B-tree indexes. **UUID v7** is time-ordered and index-friendly.
- **Snowflake**: 64 bits = timestamp (41) + machine ID (10) + sequence (12). Sortable by time, and gives millions of IDs per second. Watch for clock skew.
- **DB sequences with range allocation (hi/lo)**: each node takes a block of IDs at a time.

Q133. Explain load balancing algorithms and L4 vs L7.

- **L4** (transport layer, e.g. Azure Load Balancer): routes by IP and port. Very fast, and cannot see content.
- **L7** (application layer, e.g. Application Gateway, NGINX, Front Door): routes by path, headers or cookies. Handles TLS termination, WAF and sticky sessions.
- **Algorithms**: round robin, weighted round robin, least connections, least response time, IP hash / consistent hashing (session affinity).
- Health checks remove unhealthy instances from rotation.

Q134. How do you handle backpressure and traffic spikes?

- **Queue-based load leveling**: put a queue in front of workers and scale the workers on queue depth (**KEDA** on AKS).
- Rate limiting and load shedding (reject low-priority traffic first), and autoscaling (HPA on CPU or custom metrics).
- Bounded thread pools and queues so memory cannot blow up, plus circuit breakers.
- Pre-scale for known events (sales), and use a CDN for static content.

9. Apache Kafka

Q135. Explain Kafka's architecture.

Kafka is a **distributed, partitioned, replicated commit log** used for event streaming.

- **Broker**: a Kafka server. A **cluster** has many brokers.
- **Topic**: a named stream of records. It is split into **partitions**, which are ordered, append-only logs.
- **Offset**: the sequential position of a record within its partition.
- **Replication**: each partition has one **leader** and several **followers**. The **ISR** (in-sync replicas) are the followers that are caught up with the leader.
- **Producers** write to the leader of a partition. **Consumers** in a **consumer group** split the partitions between them.
- **Controller**: manages the cluster metadata. **KRaft** mode (the Raft protocol) replaced **ZooKeeper**. KRaft has been production-ready since 3.3, and Kafka 4.0 removed ZooKeeper completely.
- Kafka is fast because of **sequential disk I/O**, the **OS page cache**, **zero-copy** (`sendfile`), batching and compression.

Q136. How do partitions and consumer groups work? How does Kafka guarantee ordering?

- **Ordering is guaranteed only within a partition**. Records with the **same key** go to the same partition (by default `hash(key) % numPartitions`). So key by `orderId` to keep all events for one order in order.
- Within a consumer group, **each partition is assigned to exactly one consumer**. Extra consumers beyond the number of partitions sit idle, so **the partition count caps parallelism**.
- Different consumer groups each receive **all** messages independently (pub/sub).
- **Increasing the partition count** changes the key-to-partition mapping and breaks ordering for existing keys. Plan the count up front.

Q137. Explain delivery semantics: at-most-once, at-least-once, exactly-once.

- **At-most-once**: commit the offset **before** processing. A crash loses messages.
- **At-least-once** (the most common): process first, **then** commit. A crash causes duplicates, so **consumers must be idempotent**.
- **Exactly-once (EOS)**: an **idempotent producer** (`enable.idempotence=true`, which uses producer ID and sequence numbers to drop duplicates on retry) plus **transactions** (`transactional.id`) plus consumers with `isolation.level=read_committed`. It covers read-process-write **within Kafka** (e.g. Kafka Streams with `processing.guarantee=exactly_once_v2`).
- For side effects **outside** Kafka (DB writes, REST calls), you still need idempotency or the outbox/inbox patterns.

Q138. Which producer configurations matter for reliability?

- `acks=all`: the leader waits for all ISR replicas. Combine it with `min.insync.replicas=2` and `replication.factor=3`, so writes survive one broker failure.

- `enable.idempotence=true` (the default since Kafka 3.0), `retries` (effectively unlimited), `delivery.timeout.ms`.
- `max.in.flight.requests.per.connection <= 5` keeps ordering when idempotence is on.
- Throughput: `batch.size`, `linger.ms` (e.g. 5-20 ms), `compression.type=lz4` or `zstd`.
- `unclean.leader.election.enable=false`, so a replica that is out of sync never becomes leader and loses data.

Q139. Explain consumer offsets, commits and rebalancing.

- Committed offsets are stored in the internal topic `__consumer_offsets`.
- **Auto commit** (`enable.auto.commit=true`, every 5 s) can lose or duplicate messages. Prefer **manual commits** after processing (`commitSync` or `commitAsync`, or Spring's `AckMode.MANUAL` / `RECORD` / `BATCH`).
- A **rebalance** happens when consumers join or leave, or when `max.poll.interval.ms` is exceeded (slow processing). Partitions are reassigned. With eager rebalancing, all consumption stops during it.
- **Improvements**: the **CooperativeStickyAssignor** (incremental rebalancing), **static group membership** (`group.instance.id`) to avoid rebalances on rolling restarts, and the new consumer rebalance protocol (KIP-848).
- Tune `max.poll.records` and `max.poll.interval.ms` so that one batch always finishes processing within the interval.

Q140. How do you handle failures and poison messages? (Retry + DLT)

- A **poison pill** is a message that always fails (bad format, bad data). If you don't handle it, the consumer retries it forever and **blocks the partition**.
- Spring Kafka's `DefaultErrorHandler` retries with a backoff, then hands the record to a `DeadLetterPublishingRecoverer` which sends it to `topic.DLT`.
- **Non-blocking retries** (`@RetryableTopic`) send failed records to retry topics with increasing delays, so the main topic keeps flowing. The trade-off is that ordering is lost.
- Mark exceptions as **non-retryable** (e.g. validation or deserialisation errors), use `ErrorHandlingDeserializer`, and monitor and replay the DLT.

```
@RetryableTopic(attempts = "4", backoff = @Backoff(delay = 1000, multiplier = 2),
                exclude = ValidationException.class)
@KafkaListener(topics = "orders", groupId = "billing")
public void onOrder(OrderEvent e) { billing.process(e); }

@DltHandler
public void dlt(OrderEvent e, @Header(KafkaHeaders.EXCEPTION_MESSAGE) String err) {
    alerting.notify(e, err);
}
```

Q141. What is consumer lag and how do you fix a growing lag?

Lag = the log-end offset minus the committed offset. A growing lag means consumers can't keep up.

- **Monitor it**: Burrow, Kafka Exporter -> Prometheus, Confluent Control Center, Azure Monitor.
- **Fixes**: add consumers (up to the partition count), add partitions (with care for ordering), speed up processing (batch DB writes, async I/O, remove slow calls), tune `max.poll.records` and fetch sizes, and scale automatically on lag with **KEDA**.

Q142. Retention vs log compaction?

- **Retention** (`retention.ms` / `retention.bytes`): delete old segments by time or size. Kafka is **not** a queue, so messages stay after they are consumed and can be **replayed**.
- **Log compaction** (`cleanup.policy=compact`): keep **only the latest value per key**. A null value (**tombstone**) deletes the key. Use it for changelog and state topics such as customer profiles and KTable backing topics.

Q143. What is Schema Registry and why use Avro or Protobuf?

- Schema Registry stores versioned schemas. Messages carry only a small **schema ID** instead of the full schema.
- It enforces **compatibility** (BACKWARD, FORWARD, FULL), so producers cannot break consumers.
- Avro and Protobuf are compact binary formats with clear rules for evolving schemas (add fields with defaults, never reuse field IDs).
- Options: Confluent Schema Registry, or **Azure Schema Registry** in Event Hubs.

Q144. Kafka vs RabbitMQ vs Azure Service Bus vs Event Hubs?

- **Kafka**: a high-throughput, replayable log. Streaming, event sourcing, CDC pipelines. Consumers pull.
- **RabbitMQ**: a traditional broker with smart routing (exchanges). Messages are deleted once acknowledged. Task queues and RPC.
- **Azure Service Bus**: an enterprise broker with **queues and topics/subscriptions**, **sessions** (FIFO per session ID), **duplicate detection**, **scheduled messages**, a **built-in DLQ** and transactions. Best for **commands and business workflows**.
- **Azure Event Hubs**: big-data **event streaming** (telemetry, logs). It exposes a **Kafka-compatible endpoint**, so existing Kafka clients work by only changing configuration.

Q145. What is Kafka Streams / Kafka Connect?

- **Kafka Streams**: a Java library for stateful stream processing (joins, windowed aggregations, KStream / KTable). State is kept in RocksDB and backed up to changelog topics. It supports exactly-once, and needs no separate cluster.
- **Kafka Connect**: a framework for moving data in and out of Kafka without writing code. **Source** connectors (e.g. **Debezium CDC** from Postgres or MySQL) and **Sink** connectors (Elasticsearch, S3, Blob Storage, JDBC).

Q146. How do you produce and consume with Spring Kafka?

```
@Service
class OrderEventPublisher {
    private final KafkaTemplate<String, OrderEvent> kafka;
    OrderEventPublisher(KafkaTemplate<String, OrderEvent> kafka) { this.kafka = kafka; }

    void publish(OrderEvent e) {
        kafka.send("orders", e.orderId(), e)            // key = orderId keeps its events ordered
             .whenComplete((res, ex) -> {
                 if (ex != null) log.error("Publish failed", ex);
             });
    }
}

@KafkaListener(topics = "orders", groupId = "inventory", concurrency = "3")
public void consume(ConsumerRecord<String, OrderEvent> rec, Acknowledgment ack) {
    inventory.reserve(rec.value());   // idempotent
    ack.acknowledge();                // manual commit after success
}
```

Q147. How would you size a Kafka topic (partitions)?

- Partitions = max(target throughput / producer throughput per partition, target throughput / consumer throughput per partition). For example, 100 MB/s ÷ 10 MB/s per consumer = at least 10 partitions. Add headroom for growth.
- Too many partitions mean more open files, longer leader elections and more memory. Aim for thousands per broker at most.
- Key choice matters too: a skewed key (one huge customer) creates a **hot partition**.

10. Azure Cloud

Q148. Which Azure compute options would you choose for Java microservices?

- **Azure Kubernetes Service (AKS)**: full Kubernetes control. Best for many microservices, complex networking, service meshes and portability. You manage upgrades, node pools and add-ons.
- **Azure Container Apps**: serverless containers built on Kubernetes, **KEDA** and **Dapr**. Scales to zero with no cluster to manage. A great default for most microservice workloads.
- **Azure App Service**: PaaS for web apps (Java SE, Tomcat, JBoss). Deployment slots, simple autoscale.
- **Azure Functions**: event-driven serverless (HTTP, timers, Service Bus, Event Hubs triggers). **Durable Functions** handle orchestration.
- **Azure Spring Apps** was retired (announced 2024). Point to Container Apps or AKS as the migration path.
- **VMs / VM Scale Sets**: for lift-and-shift or special needs.

Q149. How do you manage secrets and identity in Azure?

- **Azure Key Vault**: secrets, keys and certificates, with rotation, access policies or RBAC, and soft-delete with purge protection.
- **Managed Identity** (system- or user-assigned): the app authenticates to Key Vault, SQL, Storage, Service Bus and more **without any credentials in code or config**, using `DefaultAzureCredential`.
- **AKS**: **Workload Identity** (federated with Entra ID) and the Secrets Store CSI driver to mount secrets from Key Vault.
- **Microsoft Entra ID (Azure AD)**: OAuth2 / OpenID Connect for users and apps, app registrations, app roles and scopes.

```
// Spring Cloud Azure: Key Vault secrets become Spring properties
spring.cloud.azure.keyvault.secret.property-sources[0].endpoint=https://my-kv.vault.azure.net/
// Auth uses Managed Identity through DefaultAzureCredential, so no secret is stored anywhere
```

Q150. Azure Service Bus vs Event Hubs vs Event Grid vs Storage Queues?

- **Service Bus**: enterprise messaging (commands and orders). FIFO sessions, DLQ, duplicate detection, transactions, topics with filtered subscriptions.
- **Event Hubs**: high-volume **event streaming** (millions of events per second), partitions, replay, Kafka endpoint, Capture to a data lake.
- **Event Grid**: **reactive event routing** with push delivery (e.g. "a blob was created", resource events, custom events). Lightweight notifications.
- **Storage Queues**: simple, cheap and very large queues with basic features.

Q151. How do you choose a database on Azure?

- **Azure SQL / SQL Managed Instance**: relational, with high-availability options, geo-replication and serverless tiers.
- **Azure Database for PostgreSQL / MySQL (Flexible Server)**: managed open-source databases.

- **Cosmos DB**: globally distributed, multi-model NoSQL with single-digit-millisecond latency. **Partition key** choice is critical. Throughput is measured in RU/s. Offers five consistency levels. **Change Feed** drives event-driven projections.
- **Azure Cache for Redis**: caching, sessions, rate limiting.
- **Blob Storage / Data Lake Gen2**: objects, files and analytics.

Q152. What is Azure API Management and where does it fit?

A full API gateway: **policies** (JWT validation, rate limit by key, quotas, caching, transformation, CORS), a developer portal, API versions and revisions, analytics, subscription keys and a self-hosted gateway. It usually sits in front of AKS or Container Apps, behind **Front Door** (global load balancing, WAF and CDN) or an **Application Gateway** (regional L7 load balancer + WAF).

Q153. How do you monitor Java apps on Azure?

- **Azure Monitor + Application Insights**: the **Java agent** (codeless auto-instrumentation built on OpenTelemetry) collects requests, dependencies, exceptions, traces and live metrics, and draws an **application map**.
- **Log Analytics workspace**: query with **KQL**. Container Insights for AKS.
- Alerts, action groups, Workbooks, and **Azure Managed Grafana / Managed Prometheus**.

```
// KQL: p95 latency per operation over the last hour
requests
| where timestamp > ago(1h)
| summarize p95 = percentile(duration, 95), failures = countif(success == false) by name
| order by p95 desc
```

Q154. How do you design a secure network in Azure?

- A hub-and-spoke **VNet** topology with **Azure Firewall** in the hub.
- **Private Endpoints** for PaaS services (SQL, Key Vault, Storage), with public access disabled.
- **NSGs** and Kubernetes network policies, **WAF** on Front Door or App Gateway, and **DDoS Protection**.
- Least-privilege **RBAC**, **Azure Policy** for governance, and **Defender for Cloud** for posture management.

Q155. How do you autoscale on Azure?

- **AKS**: **HPA** (pods, by CPU, memory or custom metrics), the **Cluster Autoscaler** (nodes), and **KEDA** (event-driven: Service Bus queue length, Kafka lag, Event Hubs). Node Auto-Provisioning (Karpenter-based) is also available.
- **Container Apps**: built-in KEDA scale rules, including scale to zero.
- **App Service**: autoscale rules on metrics or schedules.
- **Cosmos DB**: autoscale RU/s. **Azure SQL**: serverless or elastic pools.

11. DevOps, CI/CD, Docker & Kubernetes

Q156. Describe a CI/CD pipeline you have built for Java microservices.

CI (on every pull request and merge):

1. Checkout -> build with Maven or Gradle (cache dependencies).

2. Unit tests + integration tests (Testcontainers), with a JaCoCo coverage gate.

3. Static analysis: **SonarQube** quality gate, Checkstyle, SpotBugs.

4. Security: **SCA** (OWASP Dependency-Check, Snyk, Dependabot), **SAST** (CodeQL), secrets scanning.

5. Build the Docker image (Jib, Buildpacks or a multi-stage Dockerfile), **scan the image** (Trivy, Defender for Containers), and push it to **Azure Container Registry** tagged with the git SHA.

CD:

6. Deploy to dev, then test, then prod with Helm charts or Kustomize (or GitOps with **Argo CD / Flux**).

7. Smoke tests, contract tests and performance tests (JMeter, Gatling, k6) at the right stages.

8. **Approval gates** for production, **progressive delivery** (canary or blue-green), automatic rollback when health or metrics degrade.

- Tools: **Azure DevOps Pipelines** (YAML multi-stage, environments, service connections with workload identity federation) or **GitHub Actions** (OIDC to Azure, no stored secrets).

```
# Azure DevOps (simplified)
stages:
- stage: Build
  jobs:
  - job: build
    steps:
    - task: Maven@4
      inputs: { goals: 'verify', options: '-B' }
    - task: Docker@2
      inputs: { command: buildAndPush, repository: orders-svc,
                containerRegistry: acr-conn, tags: '$(Build.SourceVersion)' }
- stage: DeployProd
  dependsOn: Build
  jobs:
  - deployment: prod
    environment: prod        # approvals and checks are configured on the environment
    strategy:
      runOnce:
        deploy:
          steps:
          - script: helm upgrade --install orders ./chart --set image.tag=$(Build.SourceVersion)
```

Q157. How do you write a production-grade Dockerfile for a Spring Boot app?

- **Multi-stage build**: build in a JDK image, run in a slim **JRE** image (e.g. Eclipse Temurin JRE or a distroless image).
- **Layered jar**: dependencies change less often than code, so layer caching makes rebuilds fast.
- Run as a **non-root user**, pin base image versions, add `.dockerignore`, and put no secrets in the image.
- Use container-aware JVM flags such as `-XX:MaxRAMPercentage=75`, and handle SIGTERM for graceful shutdown (`server.shutdown=graceful`).

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -B -DskipTests package && \
    java -Djarmode=tools -jar target/app.jar extract --layers --launcher --destination extracted

FROM eclipse-temurin:21-jre
RUN useradd -r -u 1001 appuser
WORKDIR /app
COPY --from=build /app/extracted/dependencies/ ./
COPY --from=build /app/extracted/spring-boot-loader/ ./
COPY --from=build /app/extracted/snapshot-dependencies/ ./
COPY --from=build /app/extracted/application/ ./
USER 1001
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75",
    "org.springframework.boot.loader.launch.JarLauncher"]
```

Q158. Explain core Kubernetes objects.

- **Pod**: the smallest deployable unit, one or more containers. **Deployment**: declarative replica sets and rolling updates. **StatefulSet**: stable identities and storage (databases, Kafka). **DaemonSet**: one pod per node (log agents).
- **Service** (ClusterIP, NodePort, LoadBalancer): a stable virtual IP and DNS name. **Ingress** / **Gateway API**: L7 routing into the cluster.
- **ConfigMap** / **Secret**: configuration. **HPA**: autoscaling. **PodDisruptionBudget**: keeps a minimum number of pods available during node maintenance.
- **Resource requests and limits**: requests drive scheduling, limits cap usage. Exceeding the memory limit gets the pod **OOMKilled**, and CPU above the limit is throttled.
- **Namespaces**, **RBAC**, **NetworkPolicy**.

Q159. Explain liveness, readiness and startup probes.

- **Liveness**: "Is the app stuck?" If it fails, **the container is restarted**. Do not include downstream dependencies (DB) here, or a DB outage will restart every pod.
- **Readiness**: "Can the app take traffic?" If it fails, **the pod is removed from the Service endpoints** but not restarted. Check critical dependencies here.
- **Startup**: gives slow-starting JVM apps time to boot before liveness checks begin.
- Spring Boot Actuator exposes `/actuator/health/liveness` and `/actuator/health/readiness`.

Q160. Deployment strategies: rolling, blue-green, canary?

- **Rolling update**: replace pods gradually (`maxSurge`, `maxUnavailable`). This is the Kubernetes default. Old and new versions run side by side, so changes must be backward compatible.
- **Blue-green**: two full environments; switch traffic all at once. Rollback is instant, but it costs double the resources. App Service **deployment slots** work this way.
- **Canary**: send a small percentage of traffic (5% -> 25% -> 100%) to the new version while watching metrics, and roll back automatically on errors. Tools: Argo Rollouts, Flagger, a service mesh.
- **Feature flags** (Azure App Configuration, LaunchDarkly) **separate deploying code from releasing features**.
- **DB migrations** (Flyway / Liquibase) must be backward compatible: **expand -> migrate -> contract**.

Q161. What is Infrastructure as Code? Terraform vs Bicep?

Infrastructure is defined in version-controlled code: reviewed, repeatable and testable.

- **Terraform**: multi-cloud with the HCL language, a **state file** (store it remotely in an Azure Storage backend with locking), plan/apply, and modules.
- **Bicep**: Azure-native DSL that compiles to ARM templates. No state file to manage, and new Azure features are available on day zero.
- Best practices: modules, separate environments (workspaces or directories), `plan` output shown on the PR, drift detection, and policy as code (Azure Policy, OPA, Checkov).

Q162. What is GitOps?

Git is the **single source of truth** for the desired state of the cluster. An operator (**Argo CD** or **Flux**, and Flux is built into AKS as an extension) pulls from git and reconciles the cluster continuously.

- Benefits: an audit trail, easy rollback (git revert), drift correction, and no cluster credentials in CI.

Q163. What DevOps metrics do you track? (DORA)

- **Deployment frequency**, **lead time for changes**, **change failure rate**, **time to restore service (MTTR)**.
- Plus SLOs and error budgets (SRE practice): if the error budget is exhausted, pause feature releases and focus on reliability.

Q164. A pod is in CrashLoopBackOff. How do you debug it?

1. `kubectl describe pod <p>`: look at the events, exit codes (137 = OOMKilled, 1 = application error), image pull errors and probe failures.

2. `kubectl logs <p> --previous`: the logs from the crashed container.

3. Check config and secrets (missing environment variables, a wrong Key Vault reference), resource limits (the JVM heap is bigger than the memory limit), and a liveness probe that fires too early (add a startup probe).

4. `kubectl exec` or `kubectl debug` (ephemeral containers) to look inside, and `kubectl get events --sort-by=.lastTimestamp`.

12. Senior-Level Scenario & Coding Questions

Q165. Write a thread-safe, generic, bounded producer-consumer without BlockingQueue.

```
class BoundedBuffer<T> {
    private final Deque<T> items = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    BoundedBuffer(int capacity) { this.capacity = capacity; }

    void put(T t) throws InterruptedException {
        lock.lock();
        try {
            while (items.size() == capacity) notFull.await();   // while, not if: spurious wakeups
            items.addLast(t);
            notEmpty.signal();
        } finally { lock.unlock(); }
    }

    T take() throws InterruptedException {
        lock.lock();
        try {
            while (items.isEmpty()) notEmpty.await();
            T t = items.removeFirst();
            notFull.signal();
            return t;
        } finally { lock.unlock(); }
    }
}
```

Q166. Common Stream coding questions (write them live).

```
// 1. Frequency of each character
Map<Character, Long> freq = s.chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
        Collectors.counting()));

// 2. First non-repeating character
Optional<Character> first = freq.entrySet().stream()
    .filter(e -> e.getValue() == 1).map(Map.Entry::getKey).findFirst();

// 3. Second highest salary
Optional<Double> second = emps.stream().map(Employee::getSalary).distinct()
    .sorted(Comparator.reverseOrder()).skip(1).findFirst();

// 4. Partition into even and odd numbers
Map<Boolean, List<Integer>> parts = nums.stream().collect(Collectors.partitioningBy(n -> n % 2 ==
    0));

// 5. Highest-paid employee per department
Map<String, Optional<Employee>> top = emps.stream().collect(Collectors.groupingBy(
    Employee::getDept, Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))));

// 6. Find duplicates
Set<Integer> seen = new HashSet<>();
Set<Integer> dups = nums.stream().filter(n -> !seen.add(n)).collect(Collectors.toSet());

// 7. Join names, comma separated
String names = emps.stream().map(Employee::getName).collect(Collectors.joining(", ", "[", "]"));
```

Q167. Production incident: latency spikes every few minutes. How do you investigate?

- **Correlate the timing** with GC logs (long full GCs?), cron or batch jobs, cache TTL expiry (stampede), autoscaling events, and DB maintenance or vacuum.
- Check thread pools and connection pools (Hikari `pending` threads), and downstream latency in distributed traces.
- Check the Kafka consumer rebalance logs, and whether pods are restarting (OOMKilled).
- Fix the root cause, add dashboards and alerts, and write a blameless postmortem with action items.

Q168. Your service must handle 10x traffic next month. What do you do?

1. **Measure the baseline**: load test (Gatling, k6) against production-like data. Find the bottleneck (CPU, DB, locks, downstream dependencies).

2. **Scale horizontally**: stateless services, HPA/KEDA, a larger cluster, more DB read replicas.

3. **Reduce load**: caching, CDN, batching, asynchronous work through queues, pagination.

4. **Protect the system**: rate limiting, circuit breakers, bulkheads, load shedding, graceful degradation.

5. **Capacity plan** the DB (connections, IOPS, partitioning), Kafka partitions and cost. Run a game day or chaos test before the event.

Q169. How do you ensure data consistency between the Order service DB and Kafka events?

Use the **Transactional Outbox**: write the order and an outbox event in one local ACID transaction. Debezium CDC or a poller publishes the outbox rows to Kafka with at-least-once delivery. Consumers deduplicate on the event ID (inbox table or unique constraint). Key messages by `orderId` so each order's events stay in order.

Q170. How do you mentor developers and drive technical decisions? (Leadership)

- Write **ADRs** (Architecture Decision Records) that give the context, the options, the decision and the consequences.
- Run design reviews and RFCs. Give **code reviews** that teach, pair programming, and brown-bag sessions.
- Set standards: coding guidelines, shared libraries and templates (a golden-path starter), CI quality gates.
- Balance tech debt against features. Put debt in the backlog, argue for it with data (incident counts, lead time), and reserve around 20% of capacity for it.
- Use the **STAR** format (Situation, Task, Action, Result) for behavioural answers, with numbers where you have them: "cut p99 latency from 1.2 s to 180 ms", "reduced cloud cost by 30%".

Q171. Rapid-fire questions to revise the night before.

- **Can we override static methods?** No. They are hidden, not overridden, and resolved at compile time.
- **Is Java pass-by-value?** Always. For objects, the reference is passed by value.
- **Why is `wait()` in Object and not Thread?** Because monitors belong to objects. Any object can be a lock.

- **What does `final`, `finally`, `finalize` mean?** A modifier, a cleanup block, and a deprecated GC hook (use Cleaner or try-with-resources instead).
- **Can a constructor be synchronized?** No. The object is not yet visible to other threads.
- **What is a marker interface?** An interface with no methods (Serializable, Cloneable). Today annotations are usually preferred.
- **What does `transient` do?** The field is skipped during serialization.
- **Difference between `@Component`, `@Service`, `@Repository`, `@Controller`?** All are stereotypes. `@Repository` adds exception translation to Spring's `DataAccessException`.
- **`@RestController` vs `@Controller`?** `@RestController` = `@Controller` + `@ResponseBody`.
- **Why can't we have a HashMap with primitive keys?** Generics need objects, so keys are autoboxed.
- **Kafka: can two consumers in the same group read the same partition?** No. Only one consumer per partition within a group.
- **Kubernetes: what gets a pod restarted, and what removes it from load balancing?** A failed liveness probe restarts it. A failed readiness probe removes it from the Service.